gammardx@gmail.com

Specifica Tecnica

Versione 1.0.0

Informazioni documento

Redattori	A. Zanella A. Savio G. Bottacin M. Stevanin
Verificatori	A. Zanella A. Savio R. Carta M. Stevanin G. Bottacin
Destinatari	GammardX T. Vardanega R. Cardin

Registro delle modifiche

Versione	Data	Autore	Verificatore	Dettaglio
1.0.0	15/03/2026	M. Stevanin	G. Bottacin	Correzioni minori, approvazione da parte di A. Zanella
0.5.2	07/03/2026	A. Zanella	G. Bottacin	Miglioramento sezione architettura
0.5.1	05/03/2026	M. Stevanin	A. Zanella	Descrizione componenti Frontend
0.5.0	05/03/2026	A. Zanella	M. Stevanin	Aggiunta sezione Data Flow (sez. 4)
0.4.4	03/03/2026	A. Zanella	G. Bottacin	Terminato sviluppo componenti Backend
0.4.3	02/03/2026	G. Bottacin	A. Zanella	Aggiornamento stato dei requisiti
0.4.2	01/03/2026	A. Zanella	M. Stevanin	Continuo sviluppo sezione Backend
0.4.1	27/02/2026	A. Zanella	M. Stevanin	Sviluppo sezione DTO Backend
0.4.0	23/02/2026	A. Savio	A. Zanella	Stesura sezione stato dei requisiti (sez. 5)
0.3.3	21/02/2026	A. Zanella	A. Savio	Descrizione componenti Frontend
0.3.2	15/02/2026	A. Zanella	A. Savio	Sviluppo diagrammi UML Backend e Frontend
0.3.1	08/02/2026	A. Savio	A. Zanella	Stesura struttura moduli sviluppati
0.3.0	06/02/2026	A. Zanella	A. Savio	Scrittura sezione architettura (sez. 3)
0.2.0	05/02/2026	A. Savio	R. Carta	Scrittura sezione tecnologie (sez. 2)
0.1.0	04/02/2026	A. Zanella	A. Savio	Scrittura introduzione (sez. 1)

Indice

GammardX

Contents

1 Introduzione **8**

1.1 Scopo del documento . . . 8

1.2 Riferimenti . . . 8

1.2.1 Riferimenti normativi . . . 8

1.2.2 Riferimenti informativi . . . 8

2 Tecnologie **10**

2.1 Framework Frontend . . . 10

2.2 Librerie Frontend . . . 10

2.3 Tecnologie di build . . . 11

2.4 Linguaggio Frontend . . . 11

2.5 Tecnologie di test . . . 11

2.6 Framework API Backend . . . 11

2.7 Server Backend . . . 12

2.8 Linguaggio Backend . . . 12

2.9 Tecnologie di deployment . . . 12

2.10 Large Language Model . . . 12

3 Architettura **13**

3.1 Architettura logica . . . 13

3.1.1 Architettura logica Backend . . . 13

3.2 Architettura di deployment . . . 15
3.2.1 Confronto con architettura a microservizi . . . 15
3.2.2 Confronto con architettura a monolite . . . 16
3.3 UML Frontend . . . 17
3.4 Architettura Frontend . . . 18
3.4.1 DTO & Entity . . . 18
3.4.1.1 LLMResponse, LLMOutcome, LLMData . . . 18
3.4.1.2 AiHistory . . . 19
3.4.1.3 Note . . . 19
3.4.2 App . . . 20
3.4.2.1 Attributi . . . 20
3.4.2.2 Metodi . . . 20
3.4.3 Top Bar . . . 21
3.4.3.1 Attributi . . . 21
3.4.3.2 Metodi . . . 21
3.4.4 LLM Bridge . . . 22
3.4.4.1 Metodi . . . 22
3.4.5 Dialog LLM . . . 22
3.4.5.1 Attributi . . . 23
3.4.5.2 Metodi . . . 23
3.4.6 File Sidebar . . . 24
3.4.6.1 Attributi . . . 24
3.4.6.2 Metodi . . . 24
3.4.7 Markdown Editor . . . 25
3.4.7.1 Attributi . . . 25
3.4.7.2 Metodi . . . 25
3.4.8 File Service . . . 26
3.4.8.1 Metodi . . . 26
3.4.9 LLM Service . . . 26
3.4.9.1 Metodi . . . 27
3.4.10 useNotesManager . . . 27

3.4.10.1 Attributi . . . 28
3.4.10.2 Metodi . . . 28
3.4.11 useLLMDialog . . . 29
3.4.11.1 Attributi . . . 29
3.4.11.2 Metodi . . . 29
3.4.12 useSidebarResize . . . 30
3.4.12.1 Attributi . . . 30
3.4.12.2 Metodi . . . 30
3.4.13 useWordsCounter . . . 31
3.4.13.1 Metodi . . . 31
3.4.14 useNavigation . . . 31
3.4.14.1 Metodi . . . 31
3.5 UML Backend . . . 32
3.6 Architettura Backend . . . 33
3.6.1 DTO & Entity . . . 33
3.6.1.1 GenerateRequest . . . 33
3.6.1.2 TranslateRequest . . . 33
3.6.1.3 SummarizeRequest . . . 34
3.6.1.4 SixHatsRequest . . . 34
3.6.1.5 ImproveRequest . . . 34
3.6.1.6 Settings . . . 35
3.6.2 LLMResult . . . 36
3.6.2.1 Attributi . . . 36
3.6.2.2 Metodi . . . 36
3.6.3 TextDocument . . . 37
3.6.3.1 Attributi . . . 37
3.6.3.2 Metodi . . . 37
3.6.4 FastAPIAdapter . . . 38
3.6.4.1 Metodi . . . 38
3.6.5 ITextProcessor . . . 39
3.6.5.1 Metodi . . . 39

3.6.6 TextProcessorService . . . 40
3.6.6.1 Attributi . . . 40
3.6.6.2 Metodi . . . 40
3.6.7 DIContainer . . . 41
3.6.7.1 Attributi . . . 41
3.6.7.2 Metodi . . . 42
3.6.8 SummarizeTextService . . . 42
3.6.8.1 Attributi . . . 42
3.6.8.2 Metodi . . . 43
3.6.9 ImproveTextService . . . 43
3.6.9.1 Attributi . . . 43
3.6.9.2 Metodi . . . 44
3.6.10 TranslateTextService . . . 44
3.6.10.1 Attributi . . . 44
3.6.10.2 Metodi . . . 45
3.6.11 AnalyzeSixHatsService . . . 45
3.6.11.1 Attributi . . . 45
3.6.11.2 Metodi . . . 46
3.6.12 GenerateTextService . . . 46
3.6.12.1 Attributi . . . 46
3.6.12.2 Metodi . . . 47
3.6.13 IResponseParser & JSONParserAdapter . . . 47
3.6.13.1 Metodi . . . 47
3.6.14 IPromptBuilder & PromptBuilderAdapter . . . 48
3.6.14.1 Metodi . . . 48
3.6.15 IHatStrategy . . . 49
3.6.15.1 Metodi . . . 49
3.6.16 ILLMProvider . . . 50
3.6.16.1 Metodi . . . 50
3.6.17 LLMClientAdapter . . . 51
3.6.17.1 Attributi . . . 51

3.6.17.2 Metodi . . . 51
3.7 Design Pattern . . . 52
3.7.1 Object Adapter . . . 52
3.7.1.1 Definizione . . . 52
3.7.1.2 Problema risolto . . . 52
3.7.1.3 Implementazione . . . 52
3.7.1.4 Vantaggi . . . 52
3.7.2 Dependency Injection . . . 53
3.7.2.1 Definizione . . . 53
3.7.2.2 Problema risolto . . . 53
3.7.2.3 Implementazione . . . 53
3.7.2.4 Vantaggi . . . 53
3.7.3 Strategy . . . 53
3.7.3.1 Definizione . . . 53
3.7.3.2 Problema risolto . . . 54
3.7.3.3 Implementazione . . . 54
3.7.3.4 Vantaggi . . . 54

4 Data Flow **55**
4.1 Architettura di Sistema . . . 55
4.2 Fasi del Data Flow - Operazione Summarize . . . 55
4.2.1 Macro-Fase 1: Request Flow (Frontend →Backend) . . . 55
4.2.2 Macro-Fase 2: Processing (Backend →LLM →Backend) . . . 56
4.2.3 Macro-Fase 3: Response Flow (Backend →Frontend) . . . 57

5 Stato dei requisiti funzionali **58**
5.1 Stato per requisito . . . 58
5.2 Grafici riassuntivi . . . 67

1 Introduzione

1.1 Scopo del documento

Il presente documento ha l'obiettivo di fornire una descrizione chiara, strutturata ed esaustiva degli aspetti tecnici fondamentali del progetto Second Brain, con particolare riferimento all'architettura del prodotto software. In particolare, il documento analizza in modo dettagliato sia i pattern architetturali sia l'architettura di deployment, offrendo una visione completa delle componenti del sistema, delle loro responsabilità, delle interazioni reciproche e della loro distribuzione nell'ambiente di esecuzione.
La Specifica Tecnica funge da riferimento centrale per le attività di progettazione, sviluppo e manutenzione del prodotto, garantendo coerenza con il *Proof of Concept*$_G$ (*PoC*$_G$) iniziale e introducendo miglioramenti volti a consolidare la maturità architetturale della soluzione.
L'analisi dell'architettura implementativa si estende fino al livello di design di dettaglio, includendo la documentazione dei design pattern architetturali adottati, nonché la motivazione delle principali scelte progettuali in relazione alle tecnologie selezionate.

1.2 Riferimenti

1.2.1 Riferimenti normativi

- **Capitolato C6 - Second Brain**
  - https://www.math.unipd.it/~tullio/IS-1/2025/Progetto/C6.pdf
- **Regolamento progetto IS**
  - https://www.math.unipd.it/~tullio/IS-1/2025/Dispense/PD1.pdf
- **Norme di progetto**
  - https://gammardx.github.io/Documents/Documenti interni/Norme di Progetto.pdf
    Ultimo accesso: 15/03/2026

1.2.2 Riferimenti informativi

- **Dispense analisi dei requisiti**
  - https://www.math.unipd.it/~tullio/IS-1/2025/Dispense/T05.pdf
- **Glossario**

- https://gammardx.github.io/Documents/Documenti interni/Glossario.pdf
  Ultimo accesso: 15/03/2026

2 Tecnologie

In questa sezione viene fornita una panoramica delle tecnologie utilizzate dal gruppo per lo sviluppo del progetto.
Le diverse soluzioni adottate sono state organizzate in categorie sulla base del ruolo che ricoprono all'interno del sistema.
Per ciascuna tecnologia viene inoltre proposta una breve descrizione, con particolare attenzione alle caratteristiche principali e alle modalità con cui è stata impiegata nel progetto.

2.1 Framework Frontend

Nome	Versione	Descrizione
React	19.2.0	*React*$_G$ è una libreria JavaScript sviluppata da Meta per la creazione di interfacce utente, in particolare Single Page Applications (*SPA*$_G$). Si basa sull'utilizzo di componenti riutilizzabili e mette a disposizione un ampio ecosistema di librerie integrabili per supportare lo sviluppo della UI.

2.2 Librerie Frontend

Nome	Versione	Descrizione
SimpleMDE	2.20.0	Editor MarkDown, ready to use, con render live HTML del testo MD scritto.
highlight.js	11.11.1	Decoratore di codice in base alla sintassi del codice. Nello sviluppo del progetto è stato utilizzato nelle sezioni di codice presenti nelle note MD.
idb-keyval	6.2.2	Libreria che permette la gestione di operazioni di accesso e modifica dei dati in un database di tipo key-value integrato nel browser. Nel progetto è stata utilizzata per la memorizzazione delle note caricate nella WebApp
Material UI	7.3.6	Libreria che permette l'utilizzo di componenti UI pre-fabbricati.

2.3 Tecnologie di build

Nome	Versione	Descrizione
Vite	7.2.6	Vite è un tool di build e sviluppo per applicazioni web moderne che sfrutta i moduli ES nativi per offrire un ambiente di sviluppo rapido e una build ottimizzata per la produzione.

2.4 Linguaggio Frontend

Nome	Versione	Descrizione
TypeScript	5.9.3	Linguaggio di programmazione sviluppato da Microsoft, che estende JavaScript aggiungendo la tipizzazione statica.
TSX	4.8.1	*Sintassi*$_G$ che estende *TypeScript*$_G$ permettendo l'uso di JSX che a sua volta consente di descrivere le interfacce utente in modo dichiarativo.

2.5 Tecnologie di test

Nome	Versione	Descrizione
Vitest	4.0.18	*Framework*$_G$ di test utilizzato lato *Frontend*$_G$ per lo sviluppo di test d'unità. Vitest si basa sul framework Vite già implementato nel progetto.
Pytest	9.0.2	*Framework*$_G$ di test utilizzato lato *Backend*$_G$ per lo sviluppo di test d'unità.

2.6 Framework API Backend

Nome	Versione	Descrizione
FastAPI	0.128.0	Web *Framework*$_G$ ad alte performance per lo sviluppo di *API*$_G$ tramite *Python*$_G$. Questa tecnologia è stata usata per far comunicare il nostro progetto con i server di Zucchetti tramiet *API*$_G$.

2.7 Server Backend

Nome	Versione	Descrizione
Uvicorn	0.40.0	Server ASGI (Asynchronous Server Gateway Interface) per applicazioni *Python*$_G$, che consente la gestione asincrona delle richieste HTTP effettuate con FastAPI.

2.8 Linguaggio Backend

Nome	Versione	Descrizione
Python	3.12.4	Linguaggio di programmazione interpretato ad alto livello, caratterizzato da una sintassi semplice e da un vasto ecosistema di librerie, adatto a molteplici ambiti applicativi.

2.9 Tecnologie di deployment

Nome	Versione	Descrizione
Docker	4.61.0	Piattaforma di virtualizzazione che consente di impacchettare applicazioni e le loro dipendenze in container leggeri e portabili, semplificando il deployment e la scalabilità delle applicazioni.

2.10 Large Language Model

Nome	Dimensione	Descrizione
Mistral-Nemo	12B	Large Language Model sviluppato da Mistral AI in collaborazione con NVIDIA, basato sulla piattaforma NeMo.

3 Architettura

3.1 Architettura logica

L'architettura adottata per lo sviluppo del progetto è ricaduta su un'**architettura client-server**. Questa architettura prevede una comunicazione tra server (il nostro backend) e client (frontend *React*$_G$) tramite *API*$_G$ REST, e nel nostro caso il backend funge anche da adapter verso i servizi *LLM*$_G$ esterni.

3.1.1 Architettura logica Backend

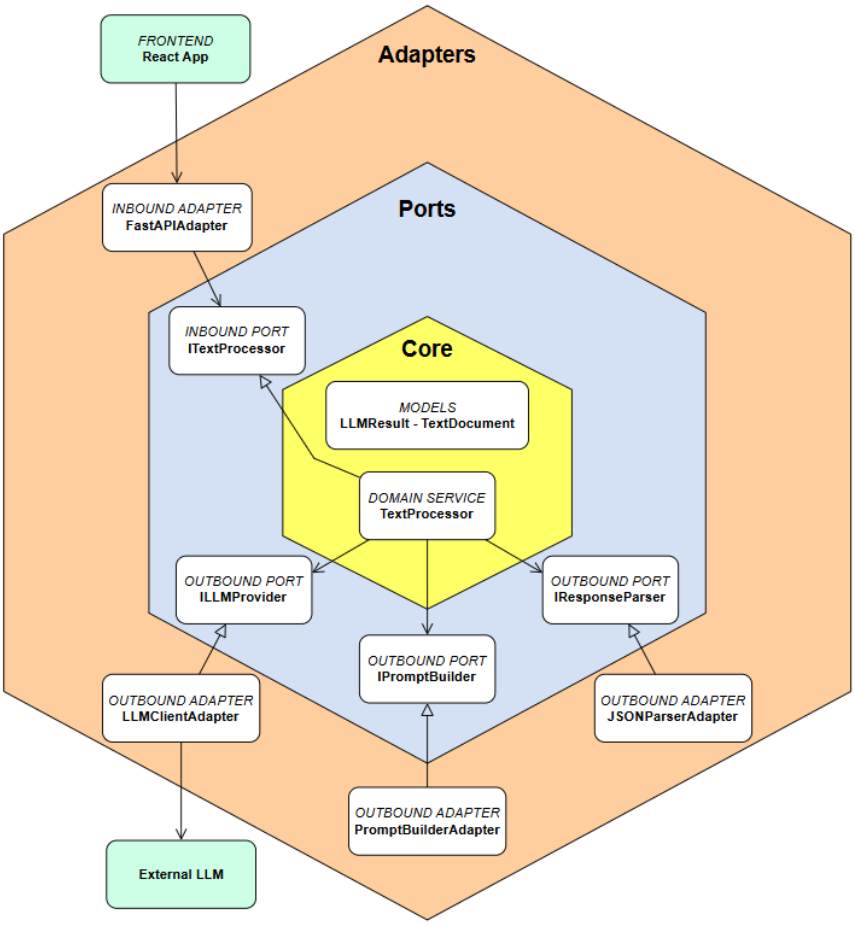

Figure 1: Architettura esagonale Backend

Il sistema è progettato seguendo l'**architettura esagonale** (Ports and Adapters), un pattern architetturale che promuove una netta separazione tra la logica di business e le interazioni con servizi esterni (LLM API), tecnologie di persistenza e interfacce utente (frontend React). Questo approccio organizza il sistema attorno a un nucleo centrale contenente il domain business, circondato da porte (interfacce) che fungono da contratti con il mondo esterno, garantendo modularità, testabilità e sostituibilità dei componenti.

Nucleo dell'Applicazione (Domain Layer)
Il nucleo dell'applicazione contiene la logica di dominio e le regole di business, progettato per essere completamente indipendente dai dettagli tecnologici esterni (framework, database, API specifiche), favorendo manutenibilità ed estendibilità del sistema.
Il nucleo non conosce FastAPI, httpx, React o tecnologie specifiche: opera esclusivamente su astrazioni.

Porte
Le porte costituiscono il punto di connessione tra il nucleo e il mondo esterno, consentendo una comunicazione strutturata attraverso interfacce ben definite.
Le Inbound Ports (Use Cases) consentono l'invocazione della logica del nucleo da parte di componenti esterni, definendo i punti di accesso all'applicazione e isolando la logica di dominio da implementazioni tecnologiche specifiche.
Le Outbound Ports permettono al nucleo di interagire con servizi esterni (LLM, database, etc.), mantenendo un'astrazione che preserva l'indipendenza della logica di business dai dettagli di implementazione.

Services (Application Layer)
I services implementano le inbound ports (Use Case Interfaces) e fanno parte della business logic applicativa, concentrandosi esclusivamente sulla logica di dominio e orchestrazione senza dipendenze tecnologiche specifiche.
Ogni service riceve le dipendenze (Secondary Ports) tramite constructor injection e orchestra le varie operazioni di: validazione, costruzione prompt tramite `IPromptBuilder`, chiamata LLM tramite `ILLMProvider`, parsing risposta tramite `IResponseParser` e il ritorno degli `LLMResult`.

Adapters (Strato Esterno)
Gli adapters rappresentano lo strato esterno del sistema e si suddividono in Input e Output Adapters.
Gli Input Adapters (Primary Adapters - Controllers) ricevono input dall'esterno e invocano le operazioni sulle porte in ingresso (Primary Ports), traducendo le richieste esterne in operazioni comprensibili per il nucleo.
Gli Output Adapters (Secondary Adapters) gestiscono la comunicazione con l'esterno

attraverso le porte in uscita, traducendo le risposte del nucleo in formati comprensibili per i servizi esterni.

Configurazione (Infrastructure Layer)
Si occupa della gestione della configurazione di sistema e della Dependency Injection.
Per la configurazione di sistema ci pensa la classe `Settings` che implementa la gestione dei file .env e la creazione dell'app FastAPI.
Mentre la Dependency Injection, delegata a la classe `DIContainer` crea e collega tutte le dipendenze, inoltre gestisce il ciclo di vita dei services

3.2 Architettura di deployment

Viste le caratteristiche del prodotto, come la dimensione e il numero di servizi, si è deciso di utilizzare un'architettura monolite che offre una scalabilità meno granulare rispetto ai microservizi, ma allo stesso tempo una complessità ridotta e più affine al tipo di progetto.
Si vuole evidenziare che l'architettura adottata è un'unione di **due monoliti** (Backend/Frontend), ognuno deployato come unità singola, che comunicano via REST. Inoltre il Backend ha un'architettura esagonale, progettato in modo tale da poter essere evoluto verso un'architettura a microservizi qualora emergessero nuovi requisiti di scalabilità o indipendenza dei servizi.
Come tecnologia di deploy è stato scelto Docker, il quale è stato utilizzato per effettuare il deploy del *Backend*$_G$ e/o del *Frontend*$_G$ in modo indipendente.

3.2.1 Confronto con architettura a microservizi

Un'architettura a microservizi avrebbe comportato la suddivisione del backend in servizi indipendenti (es: summarization-service, translation-service, improvement-service), ciascuno con proprio processo, database, e deployment indipendente.

Motivazioni per scelta a due monoliti:

1. **Semplicità operativa**: Deploy semplificato con 2 processi (frontend + backend) invece di 5-10+ servizi, eliminando necessità di orchestrazione (Kubernetes), service discovery, API gateway, e strumenti di monitoring distribuito.
2. **Performance superiore**: Comunicazione interna in-memory con latenza zero invece di chiamate HTTP inter-service (10-100ms overhead). Assenza di serializzazione/deserializzazione per comunicazioni interne.
3. **Sviluppo rapido**: Refactoring facilitato con tutto il codice in un repository. Debug semplificato senza necessità di tracciare richieste attraverso servizi distribuiti. Testing integration immediato senza orchestrazione.

4. **Scala appropriata**: Il progetto ha: un volume di richieste limitato, un team ridotto in numero, e un dominio singolo ben definito.

5. **Caratteristiche non sfruttabili dei microservizi**: Nella progettazione e nello sviluppo del progetto non si è presentata la possibilità di sfruttare i punti di forza dell'architettura a microservizi, come: la possibilità di integrare diverse tecnologie in base alle richieste di ogni servizio, o assegnare a ciascun servizio un database indipendente.

3.2.2 Confronto con architettura a monolite

Un'architettura a monolite sarebbe risultata troppo vincolante per eventuali sviluppi futuri.

Motivazioni per scelta a due monoliti:

1. **Deploy indipendenti**: permette di aggiornare frontend o backend senza dover adattare l'altra unità di deploy. In questo modo se si aggiorna solo la UI, non si rischia di introdurre problemi nel server, e viceversa.

2. **Stack tecnologici separati**: consente di utilizzare tecnologie e framework diversi tra frontend e backend in base alle esigenze di ciascuno.

3. **Scalabilità disaccoppiata**: se il backend è sotto carico (chiamate LLM costose), è possibile scalarlo indipendentemente senza dover scalare anche il frontend, che è solo statico.

4. **Testabilità separata**: come è possibile notare nel PdQ e nel MVP, adottando l'architettura a due monoliti, abbiamo potuto suddividere la fase di testing tra backend e frontend, permettendo così di sviluppare in modo indipendente le strategie di test e adottando tecnologie diverse in base alle esigenze.

5. **Sostituibilità futura**: purché il contratto REST tra backend e frontend rimanga lo stesso, in futuro sarà possibile riscrivere frontend o backend, cambiando tecnologie senza dover per forza adattare l'altra unità di deploy.

3.3 UML Frontend

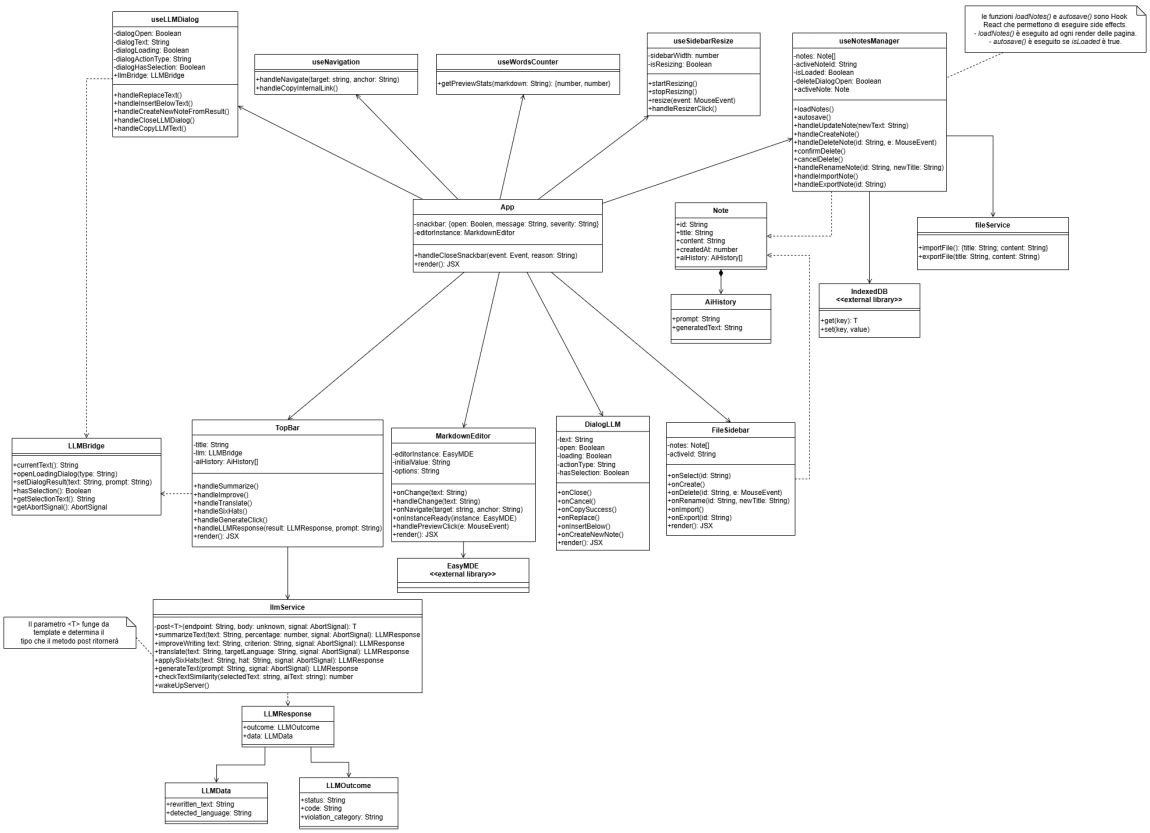

Figure 2: UML Frontend

3.4 Architettura Frontend

3.4.1 DTO & Entity

N.B.: In *TypeScript*$_G$, un'interfaccia è principalmente uno strumento di tipizzazione statica che descrive la forma di un oggetto (attributi, metodi, tipi), e viene utilizzata solo in fase di compilazione per il controllo dei tipi; non esiste a runtime e non impone realmente un'implementazione come nei linguaggi OOP classici, ma verifica solo che una struttura rispetti la forma dichiarata.

3.4.1.1 LLMResponse, LLMOutcome, LLMData

In questa sezione vengono elencate e descritte le interfacce TS utilizzate per la comunicazione tra il servizio `llmService` e il backend.
Per standardizzare le risposte generate dal *LLM*$_G$ viene utilizzata l'interfaccia `LLMResponse`, composta da altre due interfacce `LLMData` che contiene i dati (testo riscritto e lingua) prodotti dal *LLM*$_G$, e l'interfaccia `LLMOutcome` che raccoglie eventuali codici e stati d'errore emersi.

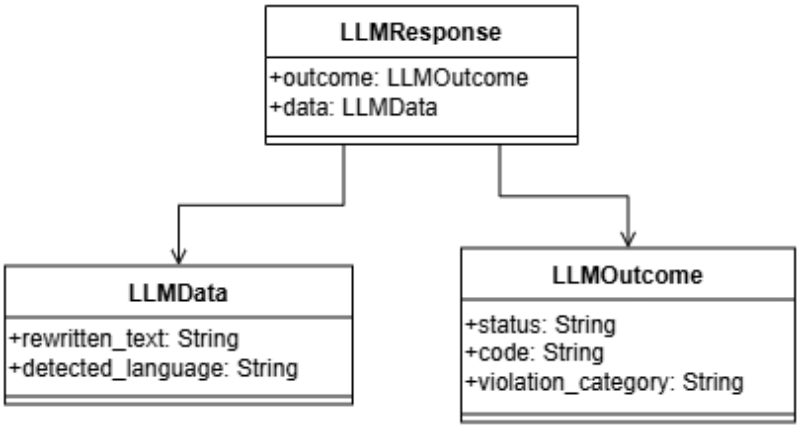

Figure 3: UML LLMResponse, LLMOutcome, LLMData

LLMResponse

- **`+outcome: LLMOutcome`**: raccoglie eventuali codici e stati d'errore emersi dalla risposta del *LLM*$_G$
- **`+data: LLMData`**: dati della risposta fornita dal *LLM*$_G$

LLMData

- **`+rewritten_text: String`**: risposta effettiva del *LLM*$_G$, contenente il testo MD riscritto

- **`+detected_language: String`**: lingua rilevata dal *LLM*$_G$ nel testo MD

LLMOutcome

- **`+status: String`**: stato della risposta con 3 possibili valori (*'success', 'refusal', 'INVALID_INPUT'*)
- **`+code: String`**: codice, d'errore o successo della risposta, generato dal *LLM*$_G$
- **`+violation_category: String`**: tipo di violazione avvenuta

3.4.1.2 AiHistory

Entità utilizzata per mantenere lo storico delle interazioni con l'*LLM*$_G$ relative a una specifica nota.

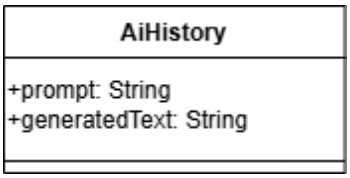

Figure 4: UML AiHistory

- **`+prompt: String`**: il prompt utilizzato dall'utente per richiedere la generazione o modifica del testo.
- **`+generatedText: String`**: il testo restituito dall'*LLM*$_G$ a seguito del prompt.

3.4.1.3 Note

Entità utilizzata per raccogliere i dati relativi alle note *Markdown*$_G$

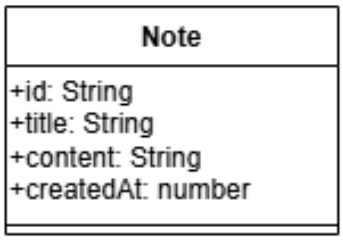

Figure 5: UML Note

- **`+id: String`**: id assegnato alla nota una volta creata/caricata a *Sistema*$_G$

- `+title: String`: titolo della nota
- `+content: String`: testo *Markdown*$_G$ contenuto nella nota
- `+createdAt: number`: timestamp di quando è stata importata/creata la nota
- `+aiHistory: AiHistory[]`: array che contiene lo storico delle interazioni AI associate a questa nota

3.4.2 App

Il componente `App` funge da radice e contenitore principale per l'interfaccia utente del frontend. Si occupa di inizializzare l'editor, gestire i messaggi di notifica globali e coordinare il layout generale.

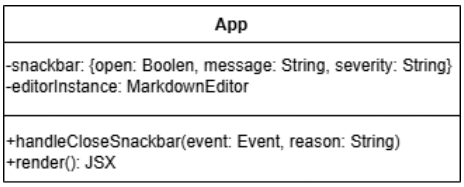

Figure 6: UML App

3.4.2.1 Attributi

- `-snackbar: {open: Boolean, message: String, severity: String}`: stato che gestisce la visibilità, il messaggio e la severità delle notifiche a schermo.
- `-editorInstance: MarkdownEditor`: riferimento all'istanza del componente di formattazione del testo.

3.4.2.2 Metodi

- `+handleCloseSnackbar(event: Event, reason: String)`: funzione per la chiusura e reset della notifica visualizzata.
- `+render(): JSX`: metodo per il rendering dell'interfaccia.

3.4.3 Top Bar

Il componente Top Bar mette a disposizione dell'utente dei bottoni per comunicare al sistema la volontà di interrogare l'LLM_G ed espone il titolo della nota che si sta visualizzando al momento.

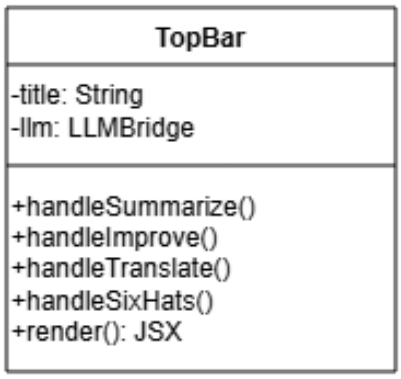

Figure 7: UML TopBar

3.4.3.1 Attributi

- `-title: String`: corrisponde al titolo della nota che si sta visualizzando.
- `-llm: LLMBridge`: fornisce l'interfaccia per le operazioni AI.
- `-aiHistory: AiHistory[]`: memorizza lo storico delle operazioni AI effettuate.

3.4.3.2 Metodi

- `+handleSummarize()`: metodo che riassume il testo presente nella nota, aprendo un dialog per far selezionare all'utente la percentuale desiderata.
- `+handleImprove()`: metodo che gestisce la funzionalità di miglioramento del testo della nota, aprendo un dialog per indicare i criteri di riscrittura.
- `+handleTranslate()`: metodo che traduce il testo MD, sfruttando un dialog per la selezione della lingua target.
- `+handleSixHats()`: metodo che si occupa della critica tramite i sei cappelli, aprendo il menu per la selezione del cappello da applicare.
- `+handleGenerateClick()`: metodo che gestisce l'apertura del dialog dedicato alla generazione di nuovo testo tramite prompt (Distant Writing).

- **`+handleLLMResponse(result: LLMResponse, prompt: String)`**: metodo che elabora il risultato restituito dal *LLM*$_G$ e aggiorna l'interfaccia.
- **`+render(): JSX`**: metodo per il rendering dell'interfaccia.

L'apertura/chiusura dei dialog avviene tramite l'utilizzo della funzione **`useState()`**, un Hook che *React*$_G$ mette a disposizione per poter assegnare variabili di stato ad un componente e gestirne i valori.

3.4.4 LLM Bridge

Classe che dichiara i metodi necessari per la comunicazione e gestione dello stato tra i dialog e le richieste verso il *LLM*$_G$.

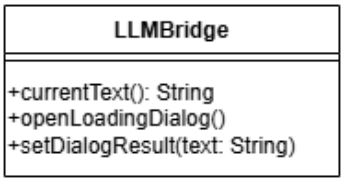

Figure 8: UML LLMBridge

3.4.4.1 Metodi

- **`+currentText(): String`**: ritorna il testo della nota al momento visualizzata.
- **`+openLoadingDialog(type: String)`**: mostra il dialog di caricamento specificando il tipo di azione in corso.
- **`+setDialogResult(text: String, prompt: String)`**: imposta il contenuto del dialog, aggiornando il testo mostrato e il prompt utilizzato.
- **`+hasSelection(): Boolean`**: verifica se l'utente ha attualmente selezionato una porzione di testo.
- **`+getSelectionText(): String`**: recupera la stringa di testo correntemente selezionata.
- **`+getAbortSignal(): AbortSignal`**: fornisce il segnale necessario per poter interrompere una richiesta in corso verso il backend.

3.4.5 Dialog LLM

Componente dedicato alla visualizzazione del risultato generato dall'*LLM*$_G$ e alle azioni che l'utente può intraprendere.

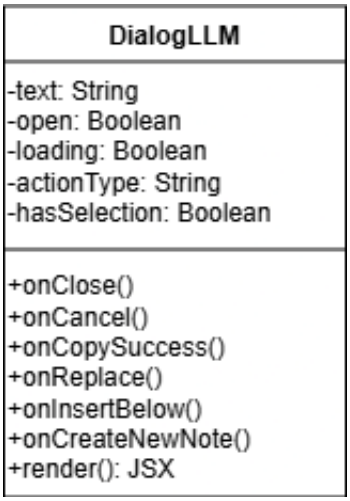

Figure 9: UML Dialog LLM

3.4.5.1 Attributi

- `-text: String`: il testo generato dall'AI da mostrare all'utente.
- `-open: Boolean`: stato che determina se il dialog è aperto e visibile.
- `-loading: Boolean`: flag che indica se è in corso un'elaborazione (chiamata di rete).
- `-actionType: String`: definisce il tipo di operazione in corso (es. riassunto, traduzione).
- `-hasSelection: Boolean`: indica se l'operazione è stata effettuata su tutto il documento o solo su una selezione.

3.4.5.2 Metodi

- `+onClose()`: gestisce la chiusura standard del dialog.
- `+onCancel()`: annulla l'operazione in corso e chiude il dialog.
- `+onCopySuccess()`: gestisce l'evento di copia del testo generato negli appunti di sistema.
- `+onReplace()`: sostituisce il testo originale (o la selezione) con il testo generato.
- `+onInsertBelow()`: inserisce il testo generato immediatamente sotto il testo originale.
- `+onCreateNewNote()`: crea una nuova nota utilizzando il testo generato come contenuto.
- `+render(): JSX`: metodo per il rendering dell'interfaccia.

3.4.6 File Sidebar

La sidebar espone le note presenti a *Sistema*$_G$, permettendo di selezionarle per visualizzarne il contenuto. Mette a disposizione dell'utente anche le funzionalità di: caricamento, scaricamento, creazione, rinomina e cancellazione delle note.

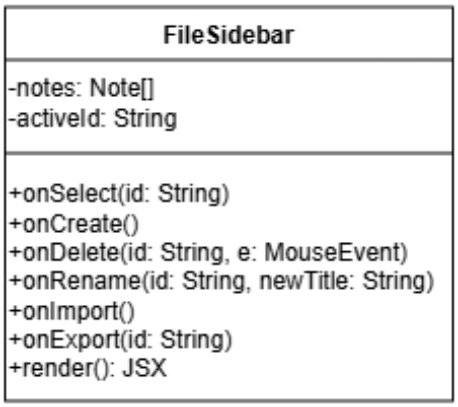

Figure 10: UML File Sidebar

3.4.6.1 Attributi

- `-notes: Note[]`: lista delle note caricate a *Sistema*$_G$
- `-activeId: String`: id della nota che sta venendo visualizzata

3.4.6.2 Metodi

- `+onSelect(id: String)`: metodo per la selezione della nota da visualizzare
- `+onCreate()`: metodo per la creazione di una nuova nota
- `+onDelete(id: String, e: MouseEvent)`: metodo per la cancellazione di una nota
- `+onRename(id: String, newTitle: String)`: metodo per la rinominazione di una nota
- `+onImport()`: metodo per il caricamento di una nota
- `+onExport(id: String)`: metodo per scaricare una nota
- `+render(): JSX`: metodo comune a tutti i moduli UI, rappresenta la funzione di **`return`** dei componenti *React*$_G$ che restituiscono del codice JSX utilizzato per descrivere la struttura dell'interfaccia utente da visualizzare.

3.4.7 Markdown Editor

Componente wrapper che incapsula la logica e l'interfaccia della libreria esterna EasyMDE, fornendo all'utente lo spazio per scrivere e formattare le note.

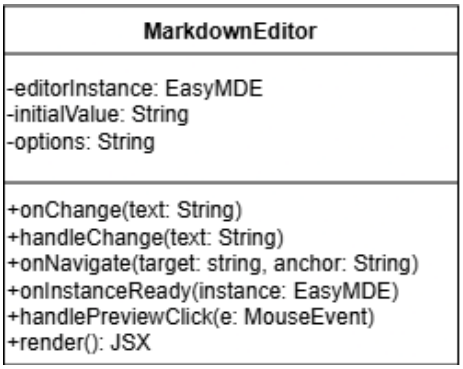

Figure 11: UML Markdown Editor

3.4.7.1 Attributi

- `-editorInstance: EasyMDE`: riferimento diretto all'istanza della libreria EasyMDE.
- `-initialValue: String`: il valore testuale di partenza caricato all'apertura della nota.
- `-options: String`: stringa (o oggetto di configurazione) contenente le opzioni di setup per EasyMDE.

3.4.7.2 Metodi

- `+onChange(text: String)`: funzione callback richiamata ai cambiamenti del testo nell'editor.
- `+handleChange(text: String)`: gestore interno per processare e propagare i cambiamenti del testo.
- `+onNavigate(target: String, anchor: String)`: gestisce la navigazione verso link interni o esterni cliccati dentro l'editor.
- `+onInstanceReady(instance: EasyMDE)`: callback eseguita una volta che l'istanza dell'editor è stata caricata completamente.
- `+handlePreviewClick(e: MouseEvent)`: gestisce la transizione tra la modalità di editing e la modalità di visualizzazione del rendering *Markdown*$_G$.
- `+render(): JSX`: metodo per il rendering dell'interfaccia.

3.4.8 File Service

Il seguente servizio gestisce il caricamento e salvataggio delle note *Markdown*$_G$ da file sistem.

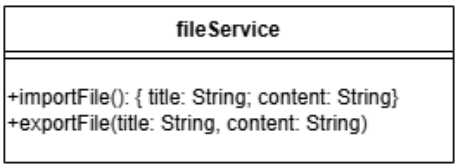

Figure 12: UML File Service

3.4.8.1 Metodi

- **`+importFile(): {title: String, content: String}`**: metodo per il caricamento di file con estensione *md* o *txt*
- **`+exportFile(title: String, content: String)`**: metodo per il salvataggio delle note presenti nel *Sistema*$_G$, in un file *md* in locale

3.4.9 LLM Service

Il servizio frontend **`llmService`** svolge il ruolo di componente di collegamento tra l'interfaccia utente e il backend applicativo. In particolare, si occupa di intercettare le richieste generate dai componenti del frontend, ad esempio operazioni di riassunto, miglioramento del testo, traduzione, distant writing o applicazione dei Sei Cappelli, e di trasformarle in un formato strutturato conforme alle *API*$_G$ esposte dal server.
Più nello specifico, il servizio provvede a:

- costruire il payload della richiesta in formato JSON, includendo il testo fornito dall'utente e i parametri specifici dell'operazione selezionata;
- inviare la richiesta al backend mediante chiamate HTTP di tipo POST verso gli endpoint definiti tramite FastAPI;
- gestire la ricezione della risposta, occupandosi del parsing del contenuto e della restituzione dei dati ai componenti dell'interfaccia utente.

In questo modo, **`llmService`** realizza un livello di astrazione che separa la logica di presentazione dalla logica di comunicazione con il backend, favorendo una maggiore modularità, manutenibilità e riusabilità del codice.

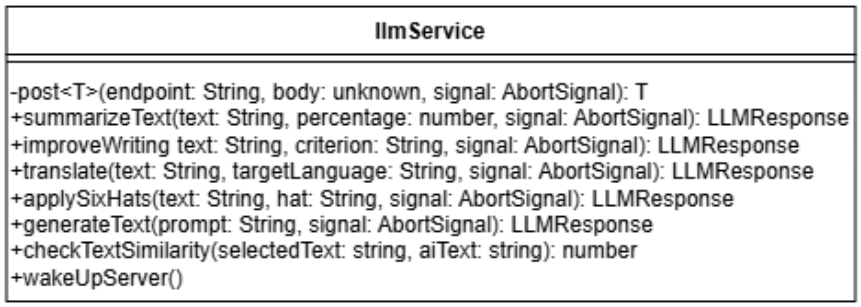

Figure 13: UML LLM Service

3.4.9.1 Metodi

- `-post<T>(endpoint: String, body: unknown, signal: AbortSignal): T`: metodo incaricato di effettuare la chiamata post al backend
- `+summarizeText(text: String, percentage: number, signal: AbortSignal): LLMResponse`: metodo utilizzato per ottenere un riassunto del testo più o meno verboso in base alla percentuale indicata
- `+improveWriting(text: String, criterion: String, signal: AbortSignal): LLMResponse`: metodo utilizzato per ottenere un testo migliorato seguendo i criteri indicati
- `+translate(text: String, targetLanguage: String, signal: AbortSignal): LLMResponse`: metodo utilizzato per ottenere la traduzione del testo in una lingua target
- `+applySixHats(text: String, hat: String, signal: AbortSignal): LLMResponse`: metodo utilizzato per ottenere una critica sul testo in base a una delle sei tipologie di critica rappresentate dai sei cappelli
- `+generateText(prompt: String, signal: AbortSignal): LLMResponse`: metodo utilizzato per ottenere del testo fornendo un prompt
- `+checkTextSimilarity(selectedText: String, aiText: String): number`: metodo utilizzato per verificare la percentuale di somiglianza tra due testi
- `+wakeUpServer()`: metodo utilizzato per accendere il server con una richiesta di salute in caso fosse entrato in sleep mode

3.4.10 useNotesManager

Hook centrale per la gestione dello stato delle note. Si occupa di caricare, salvare e sincronizzare le note con il database locale IndexedDB. Inoltre, gestisce i side effects di

React$_G$: la funzione `loadNotes()` viene eseguita ad ogni render della pagina, mentre `autosave()` viene eseguita in automatico ogni volta che `isLoaded` diventa `true`.

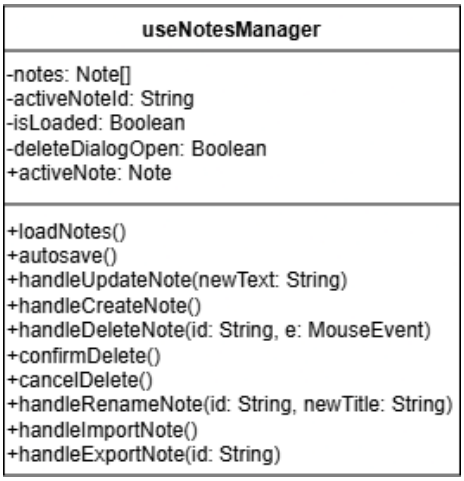

Figure 14: UML useNotesManager

3.4.10.1 Attributi

- **`-notes: Note[]`**: array contenente tutte le note caricate.
- **`-activeNoteId: String`**: ID della nota correntemente attiva nell'editor.
- **`-isLoaded: Boolean`**: flag che indica se le note sono state caricate dal DB.
- **`-deleteDialogOpen: Boolean`**: stato che gestisce la visibilità del popup di cancellazione.
- **`+activeNote: Note`**: riferimento diretto all'oggetto nota attualmente attiva.

3.4.10.2 Metodi

- **`+loadNotes()`**: carica le note dal database IndexedDB.
- **`+autosave()`**: salva automaticamente le modifiche della nota corrente.
- **`+handleUpdateNote(newText: String)`**: aggiorna il testo della nota attiva.
- **`+handleCreateNote()`**: crea una nuova nota vuota.
- **`+handleDeleteNote(id: String, e: MouseEvent)`**: prepara l'eliminazione di una nota.
- **`+confirmDelete()`** / **`+cancelDelete()`**: confermano o annullano l'eliminazione.
- **`+handleRenameNote(id: String, newTitle: String)`**: rinomina una nota esistente.

- `+handleImportNote()`: gestisce il caricamento di una nota dal file system locale.
- `+handleExportNote(id: String)`: esporta una nota sul file system locale.

3.4.11 useLLMDialog

Hook dedicato alla gestione dello stato interno e della logica del dialog utilizzato per le risposte del LLM_G.

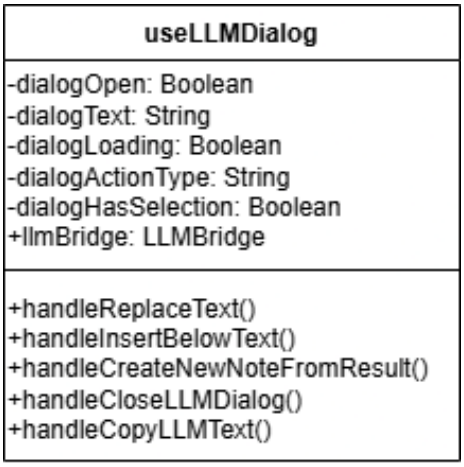

Figure 15: UML useLLMDialog

3.4.11.1 Attributi

- `-dialogOpen: Boolean`: stato di apertura del dialog.
- `-dialogText: String`: testo contenuto nel dialog.
- `-dialogLoading: Boolean`: stato di caricamento durante l'attesa della risposta AI.
- `-dialogActionType: String`: il tipo di operazione AI richiesta.
- `-dialogHasSelection: Boolean`: indica se l'azione è stata eseguita su una selezione di testo.
- `+llmBridge: LLMBridge`: riferimento all'istanza bridge per le chiamate AI.

3.4.11.2 Metodi

- `+handleReplaceText()`: sostituisce il testo nell'editor con quello del dialog.
- `+handleInsertBelowText()`: inserisce il testo del dialog sotto al testo originale.

- `+handleCreateNewNoteFromResult()`: genera una nuova nota col testo prodotto dall'AI.
- `+handleCloseLLMDialog()`: chiude il dialog e resetta lo stato.
- `+handleCopyLLMText()`: copia il testo generato negli appunti.

3.4.12 useSidebarResize

Gestisce la logica di ridimensionamento dinamico orizzontale della sidebar laterale.

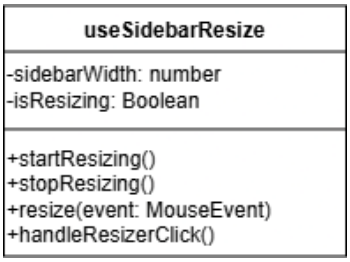

Figure 16: UML useSidebarResize

3.4.12.1 Attributi

- `-sidebarWidth: number`: la larghezza attuale della sidebar in pixel.
- `-isResizing: Boolean`: flag che indica se l'utente sta trascinando il cursore.

3.4.12.2 Metodi

- `+startResizing()` / `+stopResizing()`: gestiscono l'inizio e la fine dell'azione di ridimensionamento.
- `+resize(event: MouseEvent)`: calcola la nuova larghezza in base alla posizione del mouse.
- `+handleResizerClick()`: gestisce il click sul bordo di ridimensionamento.

3.4.13 useWordsCounter

Estrae le statistiche del testo scritto.

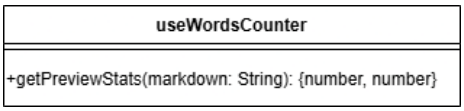

Figure 17: UML useWordsCounter

3.4.13.1 Metodi

- `+getPreviewStats(markdown: String): {number, number}`: riceve il testo markdown e restituisce un oggetto contenente il conteggio dei caratteri e delle parole.

3.4.14 useNavigation

Astrae la logica per la navigazione tra le note.

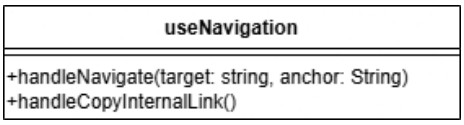

Figure 18: UML useNavigation

3.4.14.1 Metodi

- `+handleNavigate(target: String, anchor: String)`: gestisce lo spostamento verso un'altra nota o verso un'ancora specifica.
- `+handleCopyInternalLink()`: gestisce la copia negli appunti del link interno generato.

3.5 UML Backend

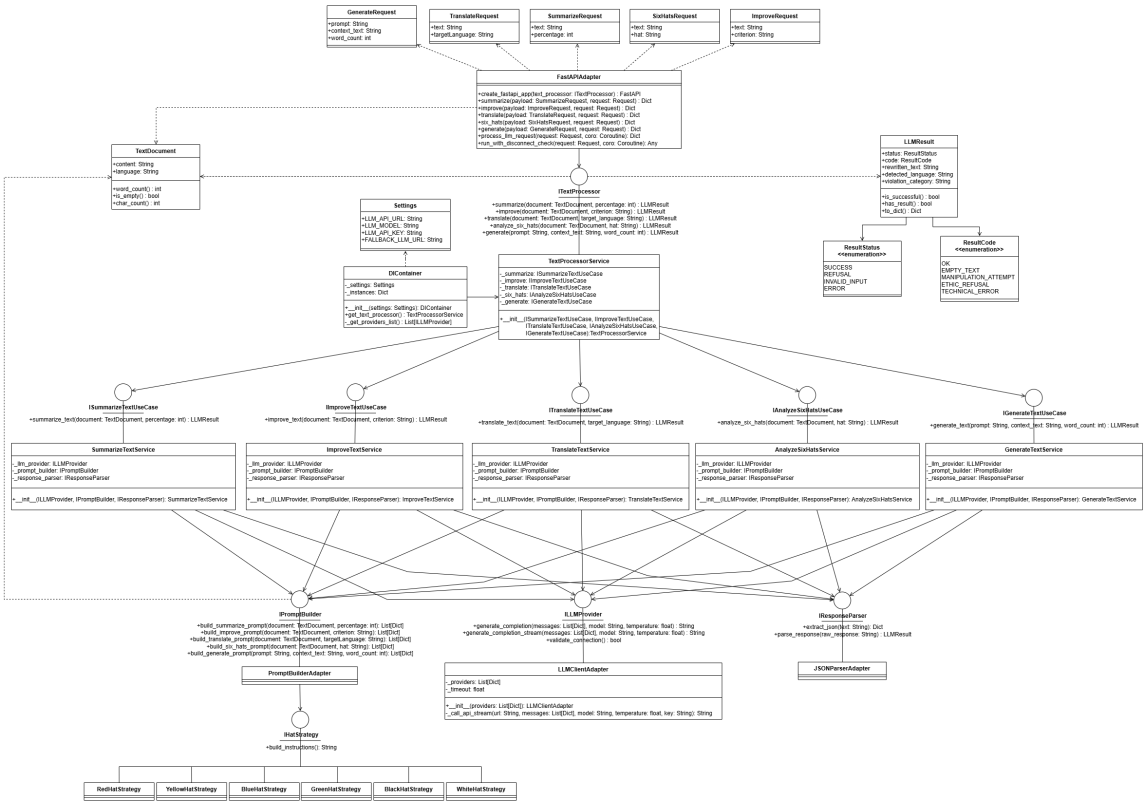

Figure 19: UML Backend

3.6 Architettura Backend

3.6.1 DTO & Entity

3.6.1.1 GenerateRequest

Il DTO in questione è utilizzato per raggruppare i parametri necessari per effettuare una richiesta di Distant Writing

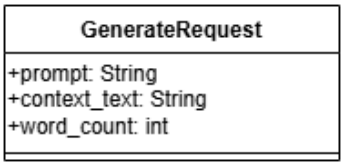

Figure 20: UML GenerateRequest

- `+prompt: String`: prompt fornito dall'utente con le richieste per il Distant Writing
- `+context_text: String`: testo selezionato dalla nota che funge da contesto per la generazione di nuovo testo, può essere una stringa vuota
- `+word_count: int`: numero di parole fornito dall'utente come lunghezza desiderata del testo che verrà generato

3.6.1.2 TranslateRequest

Il DTO in questione è utilizzato per raggruppare i parametri necessari per effettuare una richiesta di traduzione

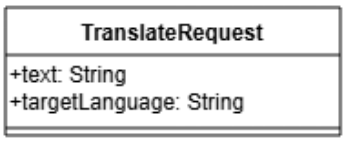

Figure 21: UML TranslateRequest

- `+text: String`: testo selezionato dall'utente che verrà tradotto
- `+targetLanguage: String`: lingua nella quale verrà tradotto il testo

3.6.1.3 SummarizeRequest

Il DTO in questione è utilizzato per raggruppare i parametri necessari per effettuare una richiesta di riassunto

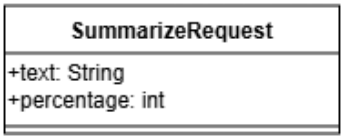

Figure 22: UML SummarizeRequest

- `+text: String`: testo selezionato dall'utente che verrà riassunto
- `+percentage: int`: valore percentuale di quanto ridurre il testo fornito

3.6.1.4 SixHatsRequest

Il DTO in questione è utilizzato per raggruppare i parametri necessari per effettuare una richiesta di critica tramite uno dei sei cappelli disponibili

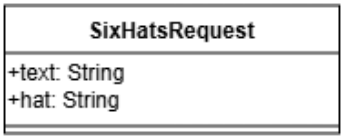

Figure 23: UML SixHatsRequest

- `+text: String`: testo selezionato dall'utente che verrà criticato
- `+hat: String`: id del cappello che verrà utilizzato per effettuare la critica

3.6.1.5 ImproveRequest

Il DTO in questione è utilizzato per raggruppare i parametri necessari per effettuare una richiesta di miglioramento

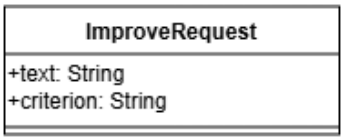

Figure 24: UML ImproveRequest

- **`+text: String`**: testo selezionato dall'utente che verrà migliorato
- **`+criterion: String`**: indicazioni fornite dall'utente in merito a come migliorare il testo

3.6.1.6 Settings

Settings è la classe responsabile della gestione della configurazione dell'applicazione. Utilizza Pydantic Settings per caricare e validare le variabili d'ambiente dal file .env

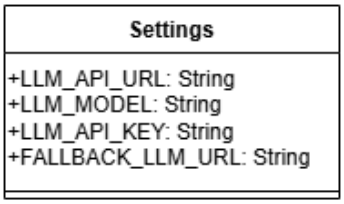

Figure 25: UML Settings

- **`+LLM`$_G$`_API_URL: String`**: URL dell'endpoint API_G del servizio LLM_G primario
- **`+LLM`$_G$`_MODEL: String`**: nome del modello LLM_G da utilizzare
- **`+LLM`$_G$`_API_KEY: String`**: chiave API_G per l'autenticazione alle richieste LLM_G
- **`+FALLBACK_LLM_URL: String`**: URL dell'API_G di fallback utilizzata in caso di errore con l'API_G primaria

3.6.2 LLMResult

Classe utilizzata per rappresentare e standardizzare le risposte fornite dal LLM_G.

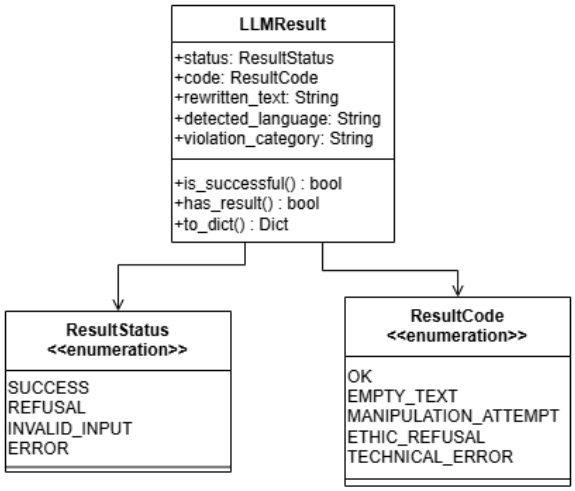

Figure 26: UML LLMResult

3.6.2.1 Attributi

- **`+status: ResultStatus`**: stato della risposta rappresentato tramite enumeration che può assumere 4 possibili valori: (*SUCCESS*, *REFUSAL*, *INVALID_INPUT*, *ERROR*)
- **`+code: ResultCode`**: codice identificativo dell'eventuale errore riscontrato con possibile valore: (*OK*, *EMPTY_TEXT*, *MANIPULATION_ATTEMPT*, *ETHIC_REFUSAL*, *TECHNICAL_ERROR*)
- **`+rewritten_text: String`**: risposta effettiva del LLM_G, contenente il testo MD riscritto
- **`+detected_language: String`**: lingua rilevata dal LLM_G nel testo MD
- **`+violation_category: String`**: descrizione testuale della ragione per cui è sorto l'eventuale errore

3.6.2.2 Metodi

- **`+is_successful(): bool`**: verifica se l'elaborazione è riuscita
- **`+has_result(): bool`**: verifica se è stato prodotto un testo dal LLM_G
- **`+to_dict(): Dict`**: converte la risposta in dizionario per serializzazione

3.6.3 TextDocument

Classe che raccoglie metodi e attributi utilizzati per la gestione del contenuto testuale delle note

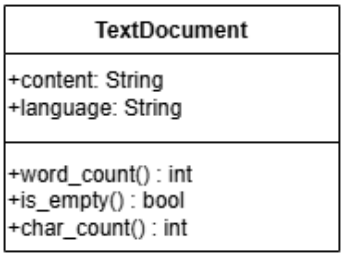

Figure 27: UML TextDocument

3.6.3.1 Attributi

- **`+content: String`**: contenuto testuale selezionato presente nella nota
- **`+language: String`**: lingua utilizzata nella nota MD selezionata

3.6.3.2 Metodi

- **`+word_count(): int`**: metodo che ritorna il numero di parole presenti nel content
- **`+is_empty(): bool`**: verifica se il testo è vuoto, e in tal caso ritorna `true`
- **`+char_count(): int`**: metodo che ritorna il numero di caratteri presenti nel content

3.6.4 FastAPIAdapter

FastAPIAdapter è un Input Adapter nell'architettura esagonale. Il suo ruolo è tradurre le richieste HTTP REST in chiamate al dominio applicativo.

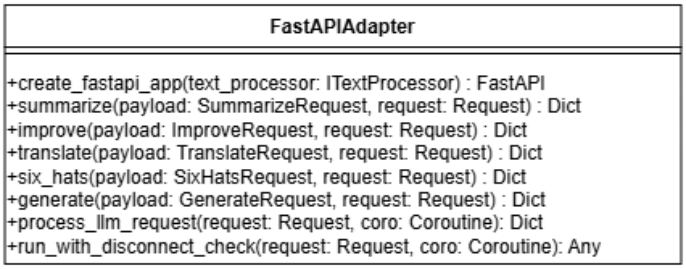

Figure 28: UML FastAPIAdapter

3.6.4.1 Metodi

- **`+create_fastapi_app(text_processor: ITextProcessor): FastAPI`**: funzione factory che crea e configura un'istanza di FastAPI. Riceve tramite dependency injection un'implementazione di `ITextProcessor` (fornita dal `DIContainer`) e registra tutti gli endpoint REST, configurando CORS, middleware e gestione degli errori. Restituisce l'applicazione FastAPI pronta per l'esecuzione.

- **`+summarize(payload: SummarizeRequest, request: Request): Dict`**: endpoint per la richiesta di riassunto che richiede un oggetto `SummarizeRequest` (DTO con i dati della richiesta), un oggetto `Request` (classe di FastAPI per la gestione di disconnessioni) e restituisce un Dict (testo JSON) contenente la risposta

- **`+improve(payload: ImproveRequest, request: Request): Dict`**: endpoint per la richiesta di miglioramento del testo, che richiede un oggetto `ImproveRequest` (DTO con i dati della richiesta), un oggetto `Request` (classe di FastAPI per la gestione di disconnessioni) e restituisce un Dict (testo JSON) contenente la risposta

- **`+translate(payload: TranslateRequest, request: Request): Dict`**: endpoint per la richiesta di traduzione che richiede un oggetto `TranslateRequest` (DTO con i dati della richiesta), un oggetto `Request` (classe di FastAPI per la gestione di disconnessioni) e restituisce un Dict (testo JSON) contenente la risposta

- **`+six_hats(payload: SixHatsRequest, request: Request): Dict`**: endpoint per la richiesta di critica del testo, che richiede un oggetto `SixHatsRequest` (DTO con i dati della richiesta), un oggetto `Request` (classe di FastAPI per la gestione di disconnessioni) e restituisce un Dict (testo JSON) contenente la risposta

- **`+generate(payload: GenerateRequest, request: Request): Dict`**: endpoint per la richiesta di generazione di testo, che richiede un oggetto `GenerateRequest` (DTO con i dati della richiesta), un oggetto `Request` (classe di FastAPI per la gestione di disconnessioni) e restituisce un Dict (testo JSON) contenente la risposta
- **`+process_llm_request(request: Request, coro: Coroutine): Dict`**: helper centrale per l'esecuzione dei task LLM_G, appoggiandosi al metodo `run_with_disconnect_check()`. Gestisce le disconnessioni, mappa le eccezioni in errori HTTP e formatta il risultato
- **`+run_with_disconnect_check()(request: Request, coro: Coroutine): Any`**: metodo che esegue la chiamata controllando ogni mezzo secondo se l'utente ha chiuso la connessione, ritornando il risultato della coroutine, oppure solleva CancelledError

3.6.5 ITextProcessor

L'interfaccia ITextProcessor è la Primary Port (porta primaria) dell'architettura esagonale. Rappresenta il contratto attraverso cui il mondo esterno (Input Adapters) comunica con il core applicativo.

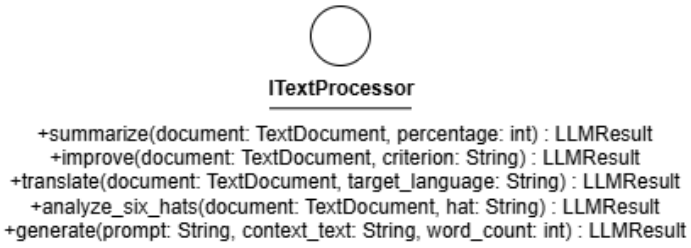

Figure 29: UML ITextProcessor

3.6.5.1 Metodi

- **`+summarize(document: TextDocument, percentage: int): LLMResult`**: definisce il contratto per riassumere un documento riducendone la lunghezza di una percentuale specificata (10-90%). Restituisce un `LLMResult` contenente il testo riassunto o un messaggio di errore.
- **`+improve(document: TextDocument, criterion: String): LLMResult`**: definisce il contratto per migliorare un documento secondo un criterio specifico (es: "chiarezza e stile professionale").
- **`+translate(document: TextDocument, target_language: String): LLMResult`**: definisce il contratto per tradurre un documento in una lingua di destinazione, rilevando

automaticamente la lingua sorgente.

- **`+analyze_six_hats(document: TextDocument, hat: String): LLMResult`**: definisce il contratto per analizzare un documento con il metodo dei sei cappelli per pensare, fornendo prospettive diverse (bianco, rosso, nero, giallo, verde, blu).
- **`+generate(prompt: String): LLMResult`**: definisce il contratto per generare testo originale basato su un prompt dell'utente.

3.6.6 TextProcessorService

`TextProcessorService` è il Domain Service che implementa l'interfaccia `ITextProcessor`, orchestrando i Use Cases dell'application layer.

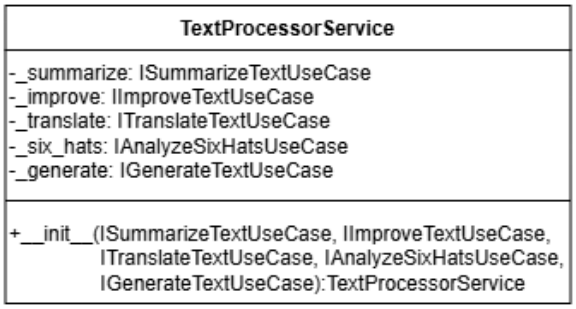

Figure 30: UML TextProcessorService

3.6.6.1 Attributi

- **`-_summarize: ISummarizeTextUseCase`**: riferimento al Use Case per riassunti
- **`-_improve: IImproveTextUseCase`**: riferimento al Use Case per miglioramenti
- **`-_translate: ITranslateTextUseCase`**: riferimento al Use Case per traduzioni
- **`-_six_hats: IAnalyzeSixHatsUseCase`**: riferimento al Use Case per analisi sei cappelli
- **`-_generate: IGenerateTextUseCase`**: riferimento al Use Case per generazione testi

3.6.6.2 Metodi

- **`+__init__(ISummarizeTextUseCase, IImproveTextUseCase, ITranslateTextUseCase, IAnalyzeSixHatsUseCase, IGenerateTextUseCase): TextProcessorService`**: costruttore che riceve i cinque Use Cases tramite Dependency Injection dal `DIContainer`.

- **`+summarize(document: TextDocument, percentage: int): LLMResult`**: implementa il contratto delegando l'esecuzione a `ISummarizeTextUseCase.summarize_text()`.
- **`+improve(document: TextDocument, criterion: String): LLMResult`**: implementa il contratto delegando a `IImproveTextUseCase.improve_text()`.
- **`+translate(document: TextDocument, target_language: String): LLMResult`**: implementa il contratto delegando a `ITranslateTextUseCase.translate_text()`.
- **`+analyze_six_hats(document: TextDocument, hat: String): LLMResult`**: implementa il contratto delegando a `IAnalyzeSixHatsUseCase.analyze_six_hats()`.
- **`+generate(prompt: String): LLMResult`**: implementa il contratto delegando a `IGenerateTextUseCase.generate_text()`.

3.6.7 DIContainer

DIContainer è la classe responsabile della creazione di tutte le dipendenze dell'applicazione. Implementa il pattern *Dependency Injection* per gestire il ciclo di vita degli oggetti e le loro interdipendenze.

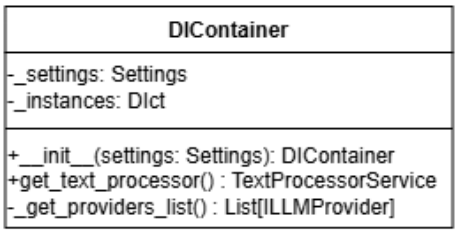

Figure 31: UML DIContainer

3.6.7.1 Attributi

- **`-_settings: Settings`**: istanza della configurazione dell'applicazione, iniettata tramite costruttore e utilizzata per configurare gli adapters con parametri come URL API_G, modello e chiavi di autenticazione.
- **`-_instances: Dict`**: cache delle istanze già create con dichiarazioni di dipendenza, permette la garanzia che esista una sola istanza nell'applicazione per i servizi come `TextProcessorService`

3.6.7.2 Metodi

- **`+__init__(settings: Settings): DIContainer`**: costruttore che inizializza il container con la configurazione validata. Memorizza `settings` per uso successivo e inizializza il dizionario `_instances` vuoto.
- **`+get_text_processor(): TextProcessorService`**: factory method principale che crea e ritorna il servizio di text processing completamente configurato. Implementa la gestione delle istanze: alla prima chiamata crea l'intera gerarchia di dipendenze (3 Output Adapters, 5 Use Cases Services, 1 Domain Service), memorizza l'istanza in cache, e alle chiamate successive ritorna la stessa istanza.
- **`-_get_providers_list(): List[ILLMProvider]`**: metodo che costruisce la lista dei provider dinamicamente in base all'ordine nel .env, che verrà poi passata al costruttore di `LLMClientAdapter`

3.6.8 SummarizeTextService

SummarizeTextService è la classe che implementa la logica applicativa per riassumere un documento testuale riducendone la lunghezza di una percentuale specificata. Implementa l'interfaccia `ISummarizeTextUseCase` che dichiara il metodo di riassunto.

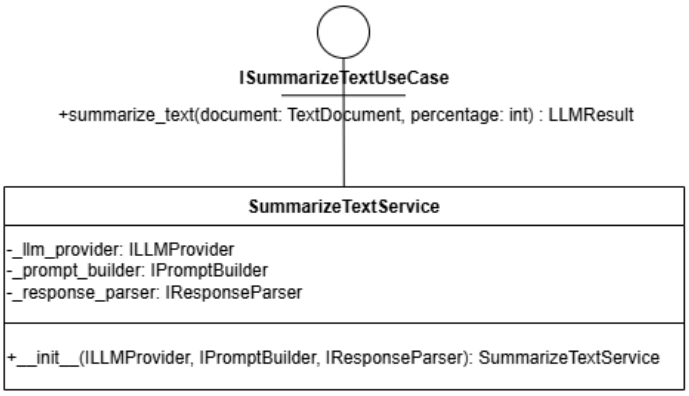

Figure 32: UML SummarizeTextService & ISummarizeTextUseCase

3.6.8.1 Attributi

- **`-_llm_provider: ILLMProvider`**: interfaccia per comunicare con il servizio LLM_G esterno, utilizzata per inviare messaggi e ricevere risposte elaborate
- **`-_prompt_builder: IPromptBuilder`**: interfaccia per costruire prompt strutturati con istruzioni di sicurezza e formato output JSON

- **`-_response_parser: IResponseParser`**: interfaccia per fare parsing delle risposte JSON (anche malformate) e convertirle in oggetti `LLMResult`

3.6.8.2 Metodi

- **`+__init__(ILLMProvider, IPromptBuilder, IResponseParser): SummarizeTextService`**: costruttore che riceve tutte le dipendenze tramite Dependency Injection. Memorizza i riferimenti nelle variabili di istanza private.

- **`+summarize_text(document: TextDocument, percentage: int): LLMResult`**: metodo principale che esegue l'operazione di riassunto, validando prima il `TextDocument` passato e delegando poi ai 3 servizi: la creazione del prompt, la chiamata al LLM_G e il parsing della risposta JSON. Infine restituisce la risposta in formato `LLMResult`

3.6.9 ImproveTextService

ImproveTextService è la classe che implementa la logica applicativa per migliorare un documento testuale applicando le richieste dell'utente. Implementa l'interfaccia `IImproveTextUseCase` che dichiara il metodo di miglioramento.

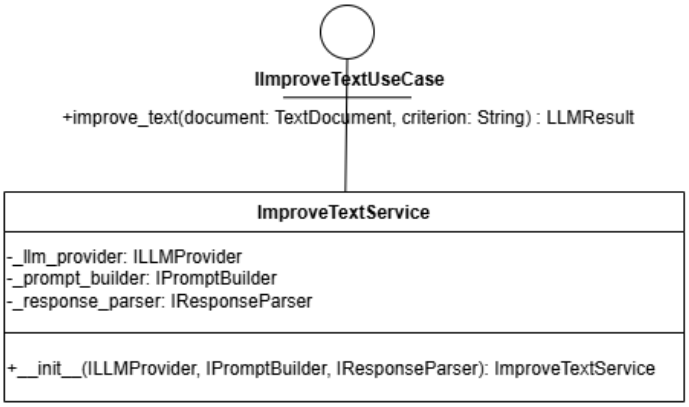

Figure 33: UML ImproveTextService & IImproveTextUseCase

3.6.9.1 Attributi

- **`-_llm_provider: ILLMProvider`**: interfaccia per comunicare con il servizio LLM_G esterno, utilizzata per inviare messaggi e ricevere risposte elaborate

- **`-_prompt_builder: IPromptBuilder`**: interfaccia per costruire prompt strutturati con istruzioni di sicurezza e formato output JSON

- **`-_response_parser: IResponseParser`**: interfaccia per fare parsing delle risposte JSON (anche malformate) e convertirle in oggetti `LLMResult`

3.6.9.2 Metodi

- **`+__init__(ILLMProvider, IPromptBuilder, IResponseParser): ImproveTextService`**: costruttore che riceve tutte le dipendenze tramite Dependency Injection. Memorizza i riferimenti nelle variabili di istanza private.

- **`+improve_text(document: TextDocument, criterion: String): LLMResult`**: metodo principale che esegue l'operazione di miglioramento, validando prima il `TextDocument` passato e delegando poi ai 3 servizi: la creazione del prompt, la chiamata al LLM_G e il parsing della risposta JSON. Infine restituisce la risposta in formato `LLMResult`

3.6.10 TranslateTextService

TranslateTextService è la classe che implementa la logica applicativa per tradurre un testo nella lingua desiderata dall'utente. Implementa l'interfaccia `ITranslateTextUseCase` che dichiara il metodo di traduzione.

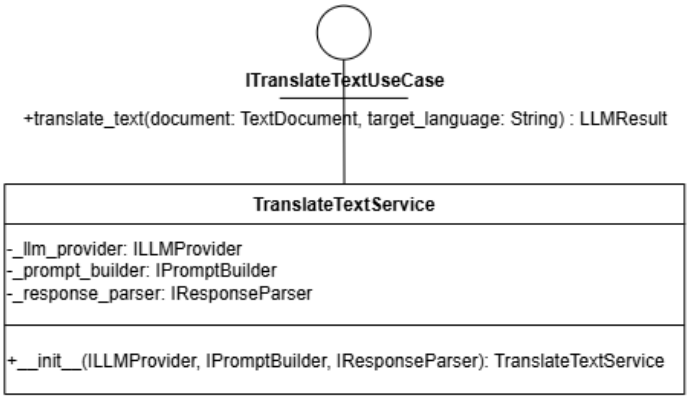

Figure 34: UML TranslateTextService & ITranslateTextUseCase

3.6.10.1 Attributi

- **`-_llm_provider: ILLMProvider`**: interfaccia per comunicare con il servizio LLM_G esterno, utilizzata per inviare messaggi e ricevere risposte elaborate

- **`-_prompt_builder: IPromptBuilder`**: interfaccia per costruire prompt strutturati con istruzioni di sicurezza e formato output JSON

- **`-_response_parser: IResponseParser`**: interfaccia per fare parsing delle risposte JSON (anche malformate) e convertirle in oggetti `LLMResult`

3.6.10.2 Metodi

- **`+__init__(ILLMProvider, IPromptBuilder, IResponseParser): TranslateTextService`**: costruttore che riceve tutte le dipendenze tramite Dependency Injection. Memorizza i riferimenti nelle variabili di istanza private.

- **`+translate_text(document: TextDocument, target_language: String): LLMResult`**: metodo principale che esegue l'operazione di traduzione, validando prima il `TextDocument` passato e delegando poi ai 3 servizi: la creazione del prompt, la chiamata al LLM_G e il parsing della risposta JSON. Infine restituisce la risposta in formato `LLMResult`

3.6.11 AnalyzeSixHatsService

AnalyzeSixHatsService è la classe che implementa la logica applicativa per criticare un testo tramite una delle sei tipologie di critica, definite dai sei cappelli di Edward De Bono. Implementa l'interfaccia `IAnalyzeSixHatsUseCase` che dichiara il metodo di critica.

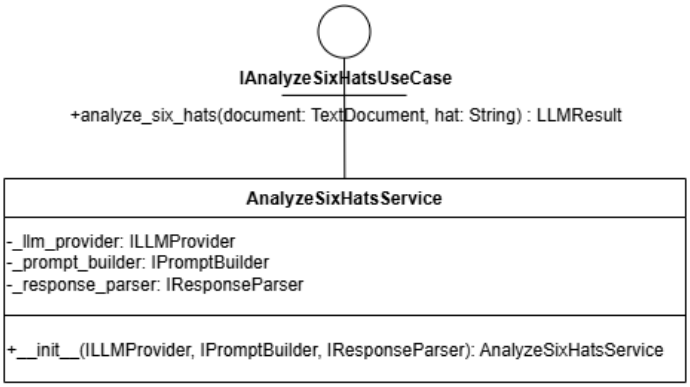

Figure 35: UML AnalyzeSixHatsService & IAnalyzeSixHatsUseCase

3.6.11.1 Attributi

- **`-_llm_provider: ILLMProvider`**: interfaccia per comunicare con il servizio LLM_G esterno, utilizzata per inviare messaggi e ricevere risposte elaborate

- **`-_prompt_builder: IPromptBuilder`**: interfaccia per costruire prompt strutturati con istruzioni di sicurezza e formato output JSON

- **`-_response_parser: IResponseParser`**: interfaccia per fare parsing delle risposte JSON (anche malformate) e convertirle in oggetti `LLMResult`

3.6.11.2 Metodi

- **`+__init__(ILLMProvider, IPromptBuilder, IResponseParser): AnalyzeSixHatsService`**: costruttore che riceve tutte le dipendenze tramite Dependency Injection. Memorizza i riferimenti nelle variabili di istanza private.

- **`+analyze_six_hats(document: TextDocument, hat: String): LLMResult`**: metodo principale che esegue l'operazione di critica, validando prima il `TextDocument` passato e delegando poi ai 3 servizi: la creazione del prompt, la chiamata al LLM_G e il parsing della risposta JSON. Infine restituisce la risposta in formato `LLMResult`

3.6.12 GenerateTextService

GenerateTextService è la classe responsabile dell'implementazione della logica applicativa necessaria alla generazione di testo a partire da un prompt fornito dall'utente, includendo anche i parametri relativi alla dimensione o alla lunghezza desiderata del testo generato. Implementa l'interfaccia `IGenerateTextUseCase` che dichiara il metodo di distant writing.

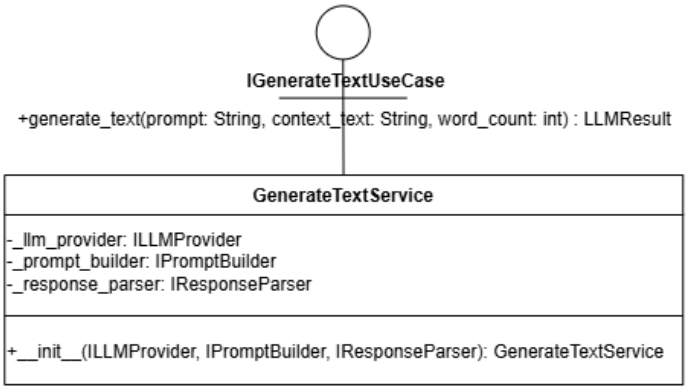

Figure 36: UML GenerateTextService & IGenerateTextUseCase

3.6.12.1 Attributi

- **`-_llm_provider: ILLMProvider`**: interfaccia per comunicare con il servizio LLM_G esterno, utilizzata per inviare messaggi e ricevere risposte elaborate

- **`-_prompt_builder: IPromptBuilder`**: interfaccia per costruire prompt strutturati con istruzioni di sicurezza e formato output JSON

- **`-_response_parser: IResponseParser`**: interfaccia per fare parsing delle risposte JSON (anche malformate) e convertirle in oggetti `LLMResult`

3.6.12.2 Metodi

- **`+__init__(ILLMProvider, IPromptBuilder, IResponseParser): GenerateTextService`**: costruttore che riceve tutte le dipendenze tramite Dependency Injection. Memorizza i riferimenti nelle variabili di istanza private.

- **`+generate_text(prompt: TextDocument, context_text: String, word_count: int): LLMResult`**: metodo principale che esegue l'operazione di scrittura del testo, validando prima il prompt passato e delegando poi ai 3 servizi: la creazione del prompt, la chiamata al LLM_G e il parsing della risposta JSON. Infine restituisce la risposta in formato `LLMResult`

3.6.13 IResponseParser & JSONParserAdapter

`IResponseParser` è un'interfaccia, utilizzata come porta dall'architettura esagonale, che definisce il contratto per il parsing e la validazione delle risposte JSON ricevute dai servizi LLM_G. Gestisce risposte malformate convertendole in oggetti del dominio. La classe `JSONParserAdapter` implementa l'interfaccia `IResponseParser` e permette la corretta applicazione del design pattern Adapter.

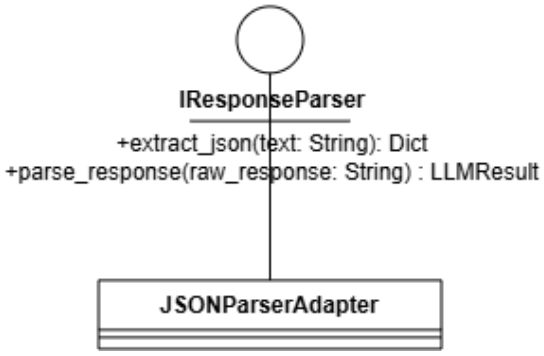

Figure 37: UML IResponseParser & JSONParserAdapter

3.6.13.1 Metodi

- **`+extract_json(text: String): Dict`**: metodo di supporto che estrae JSON da testo, con gestione di eventuali errori di estrazione, sollevando `ValueError` solo se tutti i tentativi falliscono.

- **`+parse_response(raw_response: String): LLMResult`**: metodo principale che riceve una

stringa JSON raw dall'LLM_G e la converte in un oggetto `LLMResult` del dominio, valido anche in caso di errori di parsing.

3.6.14 IPromptBuilder & PromptBuilderAdapter

L'interfaccia `IPromptBuilder` è la porta dell'architettura esagonale che definisce il contratto per la costruzione di prompt strutturati destinati ai servizi LLM_G. Incapsula la logica di prompt engineering e le strategie di sicurezza per prevenire prompt injection. La classe `PromptBuilderAdapter` implementa l'interfaccia `IPromptBuilder` e permette la corretta applicazione del design pattern Adapter.

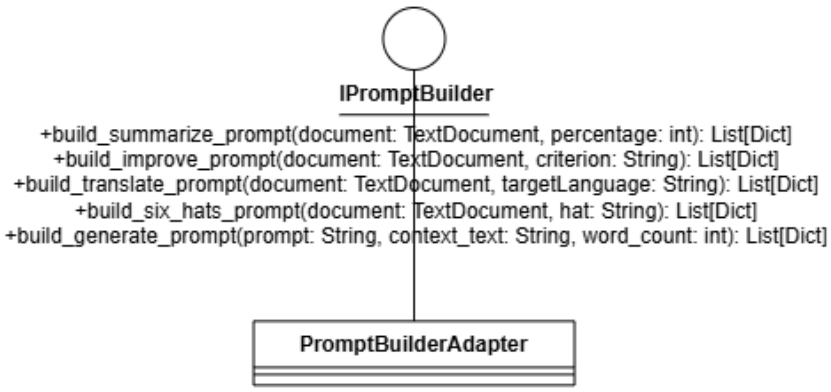

Figure 38: UML IPromptBuilder & PromptBuilderAdapter

3.6.14.1 Metodi

- **`+build_summarize_prompt(document: TextDocument, percentage: int): List[Dict]`**: costruisce i messaggi per l'operazione di riassunto. Ritorna una `List` con due `Dict`: un messaggio system dove vengono definite tutte le regole generali da rispettare dal LLM_G: es. regole anti-injection, schema JSON dell'output e un messaggio user, dove si definiscono le volontà dell'utente, quindi: testo da elaborare e percentuale target.
- **`+build_improve_prompt(document: TextDocument, criterion: String): List[Dict]`**: costruisce i messaggi per migliorare un testo secondo un criterio specifico (es: "chiarezza", "brevità", "formalità"). Ritorna una `List` con due `Dict` con due messaggi ("system", "user") come `+build_summarize_prompt()`
- **`+build_translate_prompt(document: TextDocument, target_language: String): List[Dict]`**: costruisce i messaggi per tradurre un testo, includendo istruzioni per rilevare automaticamente la lingua sorgente. Ritorna una `List` con due `Dict` con due messaggi ("system", "user") come `+build_summarize_prompt()`

- **`+build_six_hats_prompt(document: TextDocument, hat: String): List[Dict]`**: costruisce i messaggi per analisi secondo il metodo dei sei cappelli, con istruzioni specifiche per ogni prospettiva (bianco, rosso, nero, giallo, verde, blu). Ritorna una `List` con due `Dict` con due messaggi ("system", "user") come `+build_summarize_prompt()`
- **`+build_generate_prompt(prompt: String): List[Dict]`**: costruisce i messaggi per generare testo originale da un prompt utente. Ritorna una `List` con due `Dict` con due messaggi ("system", "user") come `+build_summarize_prompt()`

3.6.15 IHatStrategy

L'interfaccia `IHatStrategy` è utilizzata per implementare il pattern Strategy, fondamentale per la scelta di quale prompt di critica utilizzare.

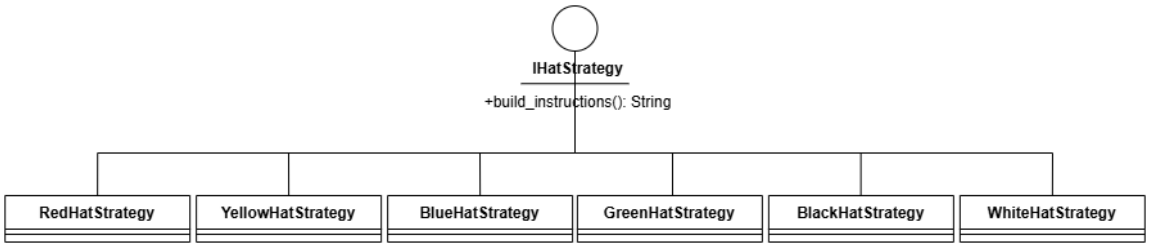

Figure 39: UML IHatStrategy & Strategy concrete

3.6.15.1 Metodi

- **`+build_instructions(): String`**: metodo implementato da ciascuna strategia concreta e a cui viene delegato la produzione dell'istruzione operativa che utilizzerà l'LLM per effettuare la critica del testo.

3.6.16 ILLMProvider

L'interfaccia `ILLMProvider` è una porta che definisce il contratto per la comunicazione con servizi LLM_G esterni. Rappresenta un'astrazione che permette ai Use Cases di richiedere elaborazioni AI senza conoscere i dettagli implementativi del provider specifico.

Figure 40: UML ILLMProvider

3.6.16.1 Metodi

- **`+generate_completion(messages: List[Dict], model: String, temperature: float): String`**: metodo asincrono che invia una lista di messaggi al servizio LLM_G e ritorna la risposta testuale completa. Come parametri vengono passati: `messages` contiene i messaggi system/user, `model` specifica il modello da utilizzare, `temperature` controlla la deterministica dell'output.
- **`+generate_completion_stream(messages: List[Dict], model: String, temperature: float): AsyncGenerator`**: variante con streaming che ritorna un generatore asincrono di testo, utilizzata per risposte in tempo reale.
- **`+validate_connection(): bool`**: metodo di verifica che controlla se la connessione al servizio LLM_G è disponibile, utile per health checks e diagnostica.

3.6.17 LLMClientAdapter

La classe `LLMClientAdapter` è un Adapter che implementa l'interfaccia `ILLMProvider`, fornendo l'implementazione concreta per la comunicazione con servizi LLM_G esterni tramite HTTP.

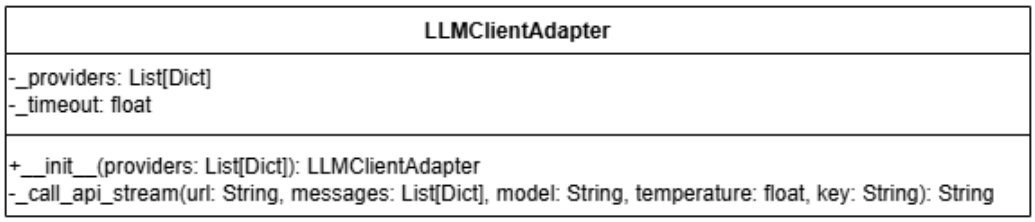

Figure 41: UML LLMClientAdapter

3.6.17.1 Attributi

- **`-_providers: List[Dict]`**: lista di tutti gli api url (anche di fallback), key e model necessari per l'elaborazione (valori provenienti da `Settings`)
- **`-_timeout: float`**: timeout HTTP impostato a 120.0 secondi per gestire elaborazioni LLM_G lunghe

3.6.17.2 Metodi

In seguito descriviamo solo i metodi non implementati dall'interfaccia `ILLMProvider`, perché già descritti tra i metodi dell'interfaccia stessa.

- **`+__init__(providers: List[Dict]): LLMClientAdapter`**: costruttore che inizializza l'adapter con la configurazione ricevuta dal `DIContainer` (valori provenienti da `Settings`)
- **`-_call_api_stream(url: String, messages: List[Dict], model: String, temperature: float, key: String): String`**: metodo privato che gestisce la comunicazione HTTP effettiva. Utilizza `httpx.AsyncClient` con metodo `stream()` per richieste POST, con request body, che oltre ai parametri del metodo, descrive i parametri `options` (num_ctx: 4096, num_gpu: 999, num_thread: 8). Infine recupera e restituisce la stringa di interesse estraendola dalla risposta del LLM_G. Ignora chunk malformati senza interrompere lo stream

3.7 Design Pattern

Il progetto fa uso di design pattern consolidati per garantire un'architettura robusta, manutenibile e testabile. Di seguito vengono analizzati tre pattern utilizzati: Object Adapter, Dependency Injection e Strategy.

3.7.1 Object Adapter

3.7.1.1 Definizione

Pattern strutturale che permette a interfacce incompatibili di collaborare, agendo come traduttore tra sistemi con interfacce diverse.

3.7.1.2 Problema risolto

Nel progetto è necessario integrare componenti con interfacce incompatibili (librerie esterne) ed isolare la Business logic come richiesto dal architettura esagonale: i Use Cases richiedono un'interfaccia `ILLMProvider`, mentre la libreria di rete `httpx` offre un'interfaccia `AsyncClient` completamente diversa. Senza un adapter, sarebbe necessario accoppiare strettamente il core applicativo ai dettagli HTTP.

3.7.1.3 Implementazione

Il pattern è implementato in quattro contesti principali:

- `PromptBuilderAdapter` adatta la logica di prompt engineering all'interfaccia `IPromptBuilder`, traducendo parametri di dominio (document, percentage) in messaggi strutturati.
- `LLMClientAdapter` traduce l'interfaccia `ILLMProvider` (attesa dai Use Cases) in chiamate HTTP con `httpx.AsyncClient`, incapsulando dettagli come streaming SSE, timeout, headers di autenticazione e fallback automatico.
- `JSONParserAdapter` adatta la logica di parsing delle risposte LLM_G, tramite metodi dichiarati nell'interfaccia `IResponseParser`.
- `FastAPIAdapter` traduce richieste HTTP FastAPI in chiamate al dominio tramite l'interfaccia `ITextProcessor`, convertendo DTO HTTP in entità di dominio e viceversa.

3.7.1.4 Vantaggi

I Use Cases rimangono completamente indipendenti dai dettagli tecnici (HTTP, JSON, streaming), il testing è facilitato dalla possibilità di iniettare mock adapters, la sostituzione di

componenti infrastrutturali richiede solo la creazione di un nuovo adapter senza modificare il core.
Inoltre la business logic è isolata e indipendente dai servizi esterni, permettendo una miglior manutenibilità e scalabilità senza doversi preoccupare dell'interfacciamento con gli altri servizi.

3.7.2 Dependency Injection

3.7.2.1 Definizione

Pattern creazionale che implementa Inversion of Control fornendo dipendenze dall'esterno invece di crearle internamente, tipicamente tramite costruttore.

3.7.2.2 Problema risolto

Componenti che creano direttamente le proprie dipendenze risultano difficili da testare, modificare e riutilizzare a causa del tight coupling. La Dependency Injection inverte il controllo: le dipendenze sono fornite dall'esterno.

3.7.2.3 Implementazione

Il progetto utilizza Constructor Injection in tutti i componenti principali. Ogni Use Case riceve le proprie dipendenze tramite costruttore: `SummarizeTextUseCase` riceve `ILLMProvider`, `IPromptBuilder`, `IResponseParser` e `model` come parametri del costruttore. Il `DIContainer` agisce come compositore centrale che crea l'intero grafo di dipendenze: istanzia gli Output Adapters (LLMClientAdapter, PromptBuilderAdapter, JSONParserAdapter), li inietta nei 5 Use Cases, e inietta i Use Cases nel `TextProcessorService`. Le dipendenze sono condivise (shared): la stessa istanza di `LLMClientAdapter` è iniettata in tutti i 5 Use Cases, ottimizzando memoria.

3.7.2.4 Vantaggi

Il testing è semplificato dalla possibilità di iniettare mock al posto di implementazioni reali, permettendo test in completo isolamento. La configurabilità è garantita dalla possibilità di iniettare implementazioni diverse per ambienti diversi (production, test, development). La manutenibilità beneficia del loose coupling: componenti disaccoppiati facilitano modifiche localizzate.

3.7.3 Strategy

3.7.3.1 Definizione

Pattern comportamentale che definisce una famiglia di algoritmi, li incapsula ciascuno in una classe separata e li rende intercambiabili, permettendo al client di variare il comportamento

indipendentemente da chi lo utilizza.

3.7.3.2 Problema risolto

La funzionalità dei Sei Cappelli richiede sei modalità distinte di composizione del prompt, ognuna con obiettivi, regole operative e stile analitico propri. L'implementazione originale concentrava tutte e sei le logiche in un dizionario statico `_HAT_INSTRUCTIONS` all'interno di `PromptBuilderAdapter`: ogni cappello era una stringa selezionata tramite chiave. Questo approccio viola il principio Open/Closed — aggiungere o modificare un cappello richiedeva la modifica diretta della classe esistente e rendeva impossibile testare ogni logica in isolamento.

3.7.3.3 Implementazione

Il pattern è strutturato in due componenti, collocati nel package `adapters/output/hat_strategies/`:

- `IHatStrategy` è l'interfaccia astratta che definisce il contratto della Strategy tramite il metodo astratto `build_instruction()`, il quale restituisce il blocco di istruzioni operative specifiche del cappello da iniettare nel system prompt.
- Le sei classi concrete — `WhiteHatStrategy`, `RedHatStrategy`, `BlackHatStrategy`, `YellowHatStrategy`, `GreenHatStrategy`, `BlueHatStrategy` — implementano ciascuna `IHatStrategy`, incapsulando la logica della propria prospettiva analitica in un file dedicato.

`PromptBuilderAdapter` funge da Context del pattern: nel costruttore istanzia le sei strategy e le raccoglie nel dizionario di istanza `_HAT_STRATEGIES`, indicizzato per nome del cappello in italiano. Il metodo `build_six_hats_prompt()` seleziona la strategy corrispondente al parametro `hat`, e delega interamente la produzione dell'istruzione operativa invocando `build_instruction()`, senza conoscere alcun dettaglio della logica interna di ciascun cappello.

3.7.3.4 Vantaggi

L'aggiunta di un settimo cappello richiede esclusivamente la creazione di una nuova classe che implementa `IHatStrategy` e la sua registrazione nel dizionario `_HAT_STRATEGIES`, senza modificare alcuna logica esistente. Ogni strategy è testabile in isolamento, indipendentemente dal ciclo di costruzione del prompt. La modifica della prospettiva analitica di un singolo cappello è localizzata in un unico file, eliminando il rischio di regressioni sul comportamento degli altri cappelli.

4 Data Flow

Il data flow descrive il percorso completo dei dati attraverso l'architettura client-server, dalla user interaction nel frontend *React*$_G$ fino ai servizi *LLM*$_G$ esterni e ritorno. Il sistema implementa una chiara separazione tra presentation layer (frontend), business logic layer (backend), e external services layer (*LLM*$_G$).

4.1 Architettura di Sistema

L'applicazione è strutturata su tre livelli principali:

1. **Client Side**: Applicazione *React*$_G$ single-page che gestisce UI, state management tramite hooks, e comunicazione HTTP con Fetch *API*$_G$
2. **Server Side**: *Backend*$_G$ *Python*$_G$ FastAPI con architettura esagonale (Adapters, Application, Domain)
3. **External Services**: Servizi *LLM*$_G$ per elaborazione linguaggio naturale

La comunicazione tra layer avviene tramite HTTP REST *API*$_G$ con payload JSON, garantendo stateless architecture e disaccoppiamento completo.

4.2 Fasi del Data Flow - Operazione Summarize

Per mostrare il Data Flow dell'applicativo useremo come esempio l'operazione di riassunto. Il flusso dati per riassumere un documento attraversa 12 fasi distinte, raggruppabili in tre macro-fasi: Request Flow (frontend →backend), Processing (backend →*LLM*$_G$ →backend), e Response Flow (backend →frontend).

4.2.1 Macro-Fase 1: Request Flow (Frontend →Backend)

FASE 1-2: User Interaction e HTTP Request
L'utente clicca il bottone "Summarize" nella UI *React*$_G$. L'event handler legge il contenuto della nota corrente dallo state *React*$_G$, valida che non sia vuoto, apre un dialog modale con loading state (spinner visibile), e costruisce la richiesta HTTP POST.
FASE 3: Backend Validation
FastAPI riceve la richiesta e esegue le operazioni di validazione e trasformazione:

1. **DTO Validation**: Pydantic valida lo schema (`text: String`, `percentage: int`)
2. **DTO →Entity**: Conversione `SummarizeRequest` (DTO adapter layer) →`TextDocument` (Entity domain layer)

Se la validazione fallisce, ritorna HTTP 422 Unprocessable Entity con dettagli dell'errore. Questo è l'unico caso dove lo status HTTP è diverso da 200.

4.2.2 Macro-Fase 2: Processing (Backend →LLM →Backend)

FASE 4-5: Domain Logic e Prompt Construction
Il `TextProcessorService` delega al `SummarizeTextUseCase` che esegue:

- **Business Validation**: *Verifica*$_G$ documento non vuoto e percentuale in range [10, 90]
- **Orchestration**: Coordina tre adapters (PromptBuilder, LLMProvider, ResponseParser)

Il `PromptBuilderAdapter` costruisce array di messaggi nel formato corretto per l'*LLM*$_G$.
FASE 6-7: LLM API Call e Processing
`LLMClientAdapter` invia richiesta HTTP al servizio *LLM*$_G$ con streaming abilitato.
Caratteristiche della chiamata:

- **Streaming**: Server-Sent Events (SSE) per risposte progressive
- **Fallback automatico**: Se *API*$_G$ primaria fallisce, tenta *API*$_G$ secondaria
- **Timeout**: 120 secondi massimi
- **GPU Optimization**: Utilizzo massimizzato risorse GPU

Il servizio *LLM*$_G$ ritorna la risposta in centinaia di chunk SSE e l'adapter accumula in una stringa JSON completa.
FASE 8: Response Parsing
`JSONParserAdapter` processa la risposta con 5 livelli di fallback progressivi per gestire JSON anche malformato:

1. Parse diretto con `json.loads()`
2. Rimozione markdown fences (“‘`json`)
3. Estrazione confini oggetto (`{...}`)
4. Sanitizzazione caratteri speciali
5. Fallback finale con error handling

Il JSON validato viene convertito in `LLMResult` (Domain Entity) con conversione stringhe →enum per `status` e `code`.

4.2.3 Macro-Fase 3: Response Flow (Backend →Frontend)

FASE 9-10: Result Return e HTTP Response

Il `LLMResult` risale attraverso i layer (Use Case →Domain Service →Input Adapter) invertendo la direzione del flusso iniziale. FastAPIAdapter invoca `result.to_dict()` per serializzare l'entità domain in dizionario JSON-compatible, che FastAPI automaticamente converte in HTTP response.

Nota importante: Lo status HTTP è sempre 200 OK. Gli errori business (validazione fallita, prompt injection, errori LLM_G) sono incapsulati nel campo `outcome.status` del payload, non nello status HTTP.

FASE 11-12: Frontend Processing e UI Update

Il frontend riceve la risposta, parsa il JSON, valida `outcome.status`, e aggiorna lo state $React_G$. Questo trigger re-render del componente `LLMDialog` che ora mostra textarea editabile con testo riassunto e bottoni "Applica" / "Annulla".

Se l'utente clicca "Applica", la nota corrente viene aggiornata nello state $React_G$, triggerando re-render di tutti i componenti dipendenti (`App`, `Sidebar`, `NoteEditor`). L'editor mostra il testo riassunto al posto dell'originale, con word count aggiornato (tipicamente riduzione 70-95%).

5 Stato dei requisiti funzionali

5.1 Stato per requisito

Codice	Descrizione	Priorità	Stato
RF-1	L'utente deve avere la possibilità di scrivere del testo di base e senza formattazione tramite editor *Markdown*$_G$	Obbligatorio	Soddisfatto
RF-2	L'utente deve avere la possibilità di applicare lo stile grassetto a sezioni di testo di sua preferenza	Obbligatorio	Soddisfatto
RF-3	L'utente deve poter inserire nuovo testo con stile grassetto nella nota	Obbligatorio	Soddisfatto
RF-4	L'utente deve poter modificare testo con stile grassetto già presente nella nota	Obbligatorio	Soddisfatto
RF-5	L'utente deve poter rimuovere lo stile grassetto da una sezione di testo nella nota	Obbligatorio	Soddisfatto
RF-6	L'utente deve avere la possibilità di applicare lo stile corsivo a sezioni di testo di sua preferenza	Obbligatorio	Soddisfatto
RF-7	L'utente deve poter inserire nuovo testo con stile corsivo nella nota	Obbligatorio	Soddisfatto
RF-8	L'utente deve poter modificare testo con stile corsivo già presente nella nota	Obbligatorio	Soddisfatto
RF-9	L'utente deve poter rimuovere lo stile corsivo da una sezione di testo nella nota	Obbligatorio	Soddisfatto
RF-10	L'utente deve avere la possibilità di applicare lo stile barrato a sezioni di testo di sua preferenza	Obbligatorio	Soddisfatto
RF-11	L'utente deve poter inserire nuovo testo con stile barrato nella nota	Obbligatorio	Soddisfatto
RF-12	L'utente deve poter modificare testo con stile barrato già presente nella nota	Obbligatorio	Soddisfatto

RF-13	L'utente deve poter rimuovere lo stile barrato da una sezione di testo nella nota	Obbligatorio	Soddisfatto
RF-14	L'utente deve essere in grado di poter inserire nella nota dei titoli con diversi livelli gerarchici	Obbligatorio	Soddisfatto
RF-15	L'utente deve poter inserire un nuovo titolo nella nota	Obbligatorio	Soddisfatto
RF-16	L'utente deve poter modificare un titolo già presente nella nota	Obbligatorio	Soddisfatto
RF-17	L'utente deve poter eliminare un titolo dalla nota	Obbligatorio	Soddisfatto
RF-18	L'utente deve avere la possibilità di inserire nella nota delle immagini, inserendo l'apposito link dell'immagine e deve poter integrare un alt opzionale relativo all'immagine	Obbligatorio	Soddisfatto
RF-19	L'utente deve poter inserire una nuova immagine nella nota	Obbligatorio	Soddisfatto
RF-20	L'utente deve poter modificare un'immagine già presente nella nota (link o alt)	Obbligatorio	Soddisfatto
RF-21	L'utente deve poter eliminare un'immagine dalla nota	Obbligatorio	Soddisfatto
RF-22	L'utente deve avere la possibilità di inserire nella nota dei link a risorse interne, come altre note, o esterne	Opzionale	Soddisfatto
RF-23	L'utente deve poter inserire un nuovo link nella nota	Opzionale	Soddisfatto
RF-24	L'utente deve poter modificare un link già presente nella nota	Opzionale	Soddisfatto
RF-25	L'utente deve poter eliminare un link dalla nota	Opzionale	Soddisfatto

RF-26	L'utente deve avere la possibilità di inserire nella nota dei link ad elementi interni alla nota stessa	Obbligatorio	Soddisfatto
RF-27	L'utente deve poter inserire una nuova ancora nella nota	Obbligatorio	Soddisfatto
RF-28	L'utente deve poter modificare un'ancora già presente nella nota	Obbligatorio	Soddisfatto
RF-29	L'utente deve poter eliminare un'ancora dalla nota	Obbligatorio	Soddisfatto
RF-30	L'utente deve avere la possibilità di inserire nella nota delle liste di elementi	Obbligatorio	Soddisfatto
RF-31	L'utente deve poter inserire un nuovo elenco nella nota	Obbligatorio	Soddisfatto
RF-32	L'utente deve poter modificare un elenco già presente nella nota	Obbligatorio	Soddisfatto
RF-33	L'utente deve poter eliminare un elenco dalla nota	Obbligatorio	Soddisfatto
RF-34	L'utente deve avere la possibilità di inserire nella nota delle sezioni di codice	Obbligatorio	Soddisfatto
RF-35	L'utente deve poter inserire una nuova sezione di codice nella nota	Obbligatorio	Soddisfatto
RF-36	L'utente deve poter modificare una sezione di codice già presente nella nota	Obbligatorio	Soddisfatto
RF-37	L'utente deve poter eliminare una sezione di codice dalla nota	Obbligatorio	Soddisfatto
RF-38	L'utente deve avere la possibilità di inserire nella nota delle tabelle	Obbligatorio	Soddisfatto
RF-39	L'utente deve poter inserire una nuova tabella nella nota	Obbligatorio	Soddisfatto
RF-40	L'utente deve poter modificare il contenuto di una cella di una tabella nella nota	Obbligatorio	Soddisfatto
RF-41	L'utente deve poter aggiungere una riga a una tabella nella nota	Obbligatorio	Soddisfatto

RF-42	L'utente deve poter aggiungere una colonna a una tabella nella nota	Obbligatorio	Soddisfatto
RF-43	L'utente deve poter rimuovere una riga da una tabella nella nota	Obbligatorio	Soddisfatto
RF-44	L'utente deve poter rimuovere una colonna da una tabella nella nota	Obbligatorio	Soddisfatto
RF-45	L'utente deve poter eliminare una tabella dalla nota	Obbligatorio	Soddisfatto
RF-46	L'utente deve avere la possibilità di inserire nella nota delle citazioni	Obbligatorio	Soddisfatto
RF-47	L'utente deve poter inserire una nuova citazione nella nota	Obbligatorio	Soddisfatto
RF-48	L'utente deve poter modificare una citazione già presente nella nota	Obbligatorio	Soddisfatto
RF-49	L'utente deve poter eliminare una citazione dalla nota	Obbligatorio	Soddisfatto
RF-50	L'utente deve avere la possibilità di caricare una nota	Obbligatorio	Soddisfatto
RF-51	L'utente deve avere la possibilità di scegliere se caricare una nota dal suo file system	Obbligatorio	Soddisfatto
RF-52	L'utente deve avere la possibilità di scegliere se caricare una nota dal *database*$_G$	Opzionale	Non soddisfatto
RF-53	L'utente deve ricevere un messaggio di errore in caso di mancato caricamento della nota	Obbligatorio	Soddisfatto
RF-54	L'utente deve avere la possibilità di salvare una nota	Obbligatorio	Soddisfatto
RF-55	L'utente deve avere la possibilità di scegliere se salvare una nota nel suo file system	Obbligatorio	Soddisfatto

RF-56	L'utente deve avere la possibilità di scegliere se salvare una nota nel *database*$_G$	Opzionale	Non soddisfatto
RF-57	L'utente deve ricevere un messaggio di errore in caso di mancato salvataggio della nota	Obbligatorio	Soddisfatto
RF-58	L'utente deve avere la possibilità di eliminare una nota	Obbligatorio	Soddisfatto
RF-59	L'utente deve avere la possibilità di scegliere se eliminare una nota nel suo file system	Obbligatorio	Soddisfatto
RF-60	L'utente deve avere la possibilità di scegliere se eliminare una nota nel *database*$_G$	Opzionale	Non soddisfatto
RF-61	L'utente deve ricevere un messaggio di errore in caso di mancata eliminazione della nota	Obbligatorio	Soddisfatto
RF-62	L'utente deve avere la possibilità di annullare l'eliminazione di una nota	Obbligatorio	Soddisfatto
RF-63	L'utente deve poter visualizzare una nota	Obbligatorio	Soddisfatto
RF-64	L'utente deve poter visualizzare una nota esclusivamente nella zona editor	Obbligatorio	Soddisfatto
RF-65	L'utente deve poter visualizzare una nota esclusivamente nella zona *render*$_G$	Obbligatorio	Soddisfatto
RF-66	L'utente deve poter visualizzare una nota in modo combinato: sia nella zona editor che in quella *render*$_G$	Obbligatorio	Soddisfatto
RF-67	L'utente deve poter riassumere il contenuto di una nota tramite *LLM*$_G$	Obbligatorio	Soddisfatto
RF-68	L'utente deve poter riassumere una selezione del contenuto di una nota tramite *LLM*$_G$	Obbligatorio	Soddisfatto
RF-69	L'utente deve poter riassumere l'intero contenuto di una nota tramite *LLM*$_G$	Obbligatorio	Soddisfatto

RF-70	L'utente deve poter ricevere un messaggio di errore in caso di una chiamata API_G all'LLM_G non andata a buon fine a seguito di una richiesta di riassunto	Obbligatorio	Soddisfatto
RF-71	L'utente deve poter annullare la richiesta di riassumere il contenuto di una nota tramite LLM_G	Obbligatorio	Soddisfatto
RF-72	L'utente deve poter salvare il riassunto dell'intero documento come nuova nota	Obbligatorio	Soddisfatto
RF-73	L'utente deve poter tradurre il contenuto di una nota tramite LLM_G	Obbligatorio	Soddisfatto
RF-74	L'utente deve poter tradurre una selezione del contenuto di una nota tramite LLM_G	Obbligatorio	Soddisfatto
RF-75	L'utente deve poter tradurre l'intero contenuto di una nota tramite LLM_G	Obbligatorio	Soddisfatto
RF-76	L'utente deve poter ricevere un messaggio di errore in caso di una chiamata API_G all'LLM_G non andata a buon fine a seguito di una richiesta di traduzione	Obbligatorio	Soddisfatto
RF-77	L'utente deve poter annullare la richiesta di traduzione del contenuto di una nota tramite LLM_G	Obbligatorio	Soddisfatto
RF-78	L'utente deve poter salvare la traduzione dell'intero documento come nuova nota	Obbligatorio	Soddisfatto
RF-79	L'utente deve poter riscrivere il contenuto di una nota tramite LLM_G	Obbligatorio	Soddisfatto
RF-80	L'utente deve poter riscrivere una selezione del contenuto di una nota tramite LLM_G	Obbligatorio	Soddisfatto
RF-81	L'utente deve poter riscrivere l'intero contenuto di una nota tramite LLM_G	Obbligatorio	Soddisfatto
RF-82	L'utente deve poter ricevere un messaggio di errore in caso di una chiamata API_G all'LLM_G non andata a buon fine a seguito di una richiesta di riscrittura	Obbligatorio	Soddisfatto

RF-83	L'utente deve poter annullare la richiesta di riscrivere il contenuto di una nota tramite LLM_G	Obbligatorio	Soddisfatto
RF-84	L'utente deve poter salvare la riscrittura dell'intero documento come nuova nota	Obbligatorio	Soddisfatto
RF-85	L'utente deve poter eseguire un'analisi critica sul contenuto di una nota tramite LLM_G	Obbligatorio	Soddisfatto
RF-86	L'utente deve poter eseguire un'analisi critica su una selezione del contenuto di una nota tramite LLM_G	Obbligatorio	Soddisfatto
RF-87	L'utente deve poter eseguire un'analisi critica sull'intero contenuto di una nota tramite LLM_G	Obbligatorio	Soddisfatto
RF-88	L'utente deve poter eseguire un'analisi in stile obiettivo sul contenuto di una nota tramite LLM_G	Obbligatorio	Soddisfatto
RF-89	L'utente deve poter eseguire un'analisi in stile emotivo sul contenuto di una nota tramite LLM_G	Obbligatorio	Soddisfatto
RF-90	L'utente deve poter eseguire un'analisi in stile ottimista sul contenuto di una nota tramite LLM_G	Obbligatorio	Soddisfatto
RF-91	L'utente deve poter eseguire un'analisi in stile critico sul contenuto di una nota tramite LLM_G	Obbligatorio	Soddisfatto
RF-92	L'utente deve poter eseguire un'analisi in stile creativo sul contenuto di una nota tramite LLM_G	Obbligatorio	Soddisfatto
RF-93	L'utente deve poter eseguire un'analisi in stile logico sul contenuto di una nota tramite LLM_G	Obbligatorio	Soddisfatto

RF-94	L'utente deve poter ricevere un messaggio di errore in caso di una chiamata API_G all'LLM_G non andata a buon fine a seguito di una richiesta di analisi critica del contenuto	Obbligatorio	Soddisfatto
RF-95	L'utente deve poter annullare la richiesta di analisi critica del contenuto di una nota tramite LLM_G	Obbligatorio	Soddisfatto
RF-96	L'utente deve poter salvare l'analisi critica del contenuto come nuova nota	Obbligatorio	Soddisfatto
RF-97	L'utente deve poter generare automaticamente il testo di una nota tramite LLM_G	Obbligatorio	Soddisfatto
RF-98	L'utente deve poter annullare la richiesta di generazione automatica del testo di una nota tramite LLM_G	Obbligatorio	Soddisfatto
RF-99	L'utente deve avere la possibilità di effettuare undo e redo riguardo le modifiche apportate al testo	Obbligatorio	Soddisfatto
RF-100	L'utente deve poter riconoscere quali sezioni di testo sono state generate dal LLM_G e quali invece sono state modificate dall'utente stesso	Desiderabile	Soddisfatto
RF-101	L'utente deve poter riutilizzare e modificare un vecchio prompt utilizzato nel distant writing, per rigenerare il testo	Desiderabile	Soddisfatto
RF-102	L'utente deve poter selezionare un capitolo con un tipo di selezione dedicata e non con il cursore	Opzionale	Non soddisfatto
RF-103	L'utente deve poter visualizzare la nota desiderata, presente tra le note nel browser	Obbligatorio	Soddisfatto
RF-104	L'utente deve poter creare una nuova nota, e deve essere accessibile tra le note nel browser	Obbligatorio	Soddisfatto

RF-105	L'utente deve poter rinominare la nota desiderata, presente tra le note nel browser	Obbligatorio	Soddisfatto
RF-106	L'utente deve poter gestire la visualizzazione della sidebar	Opzionale	Soddisfatto
RF-107	L'utente deve poter aprire/chiudere la sidebar	Opzionale	Soddisfatto
RF-108	L'utente deve poter gestire l'apertura e la chiusura della sidebar	Opzionale	Soddisfatto
RF-109	L'utente deve poter scegliere la lingua nella quale tradurre il testo	Obbligatorio	Soddisfatto
RF-110	L'utente deve poter selezionare il tipo di analisi critica da ricevere	Obbligatorio	Soddisfatto
RF-111	L'utente deve poter inserire il prompt per il distant writing in una cella di testo apposita	Obbligatorio	Soddisfatto
RF-112	L'utente deve poter selezionare la percentuale di riduzione del testo da riassumere	Obbligatorio	Soddisfatto
RF-113	L'utente deve poter specificare le preferenze di riscrittura in una casella di testo dedicata	Obbligatorio	Soddisfatto
RF-114	L'utente deve poter selezionare il numero di parole desiderato (massimo 10.000) per la generazione automatica del testo	Obbligatorio	Soddisfatto
RF-115	L'utente deve poter visualizzare le informazioni della nota (numero di caratteri e numero di parole) in tempo reale	Obbligatorio	Soddisfatto
RF-116	L'editor deve essere basato su *Markdown*$_G$ per la formattazione del testo	Obbligatorio	Soddisfatto
RF-117	Il *Sistema*$_G$ deve produrre documenti in formato *Markdown*$_G$	Obbligatorio	Soddisfatto

5.2 Grafici riassuntivi

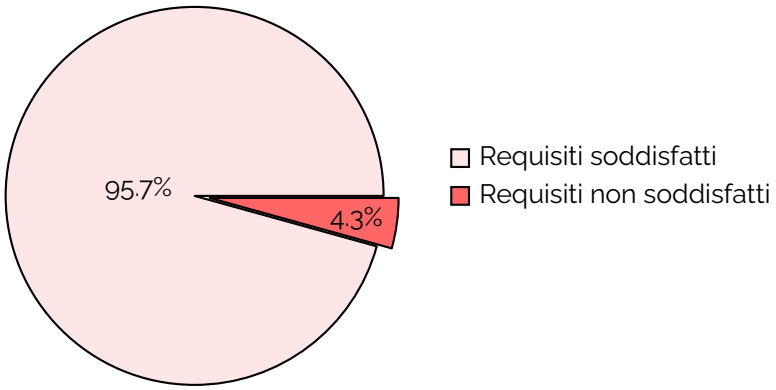

Figure 42: Percentuale di requisiti funzionali soddisfatti in totale

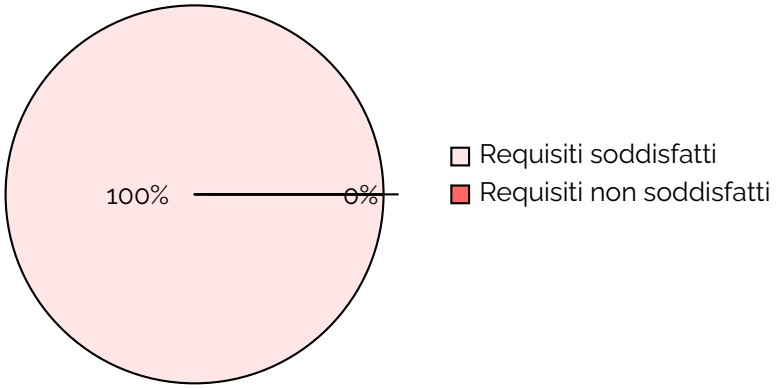

Figure 43: Percentuale di requisiti funzionali obbligatori soddisfatti

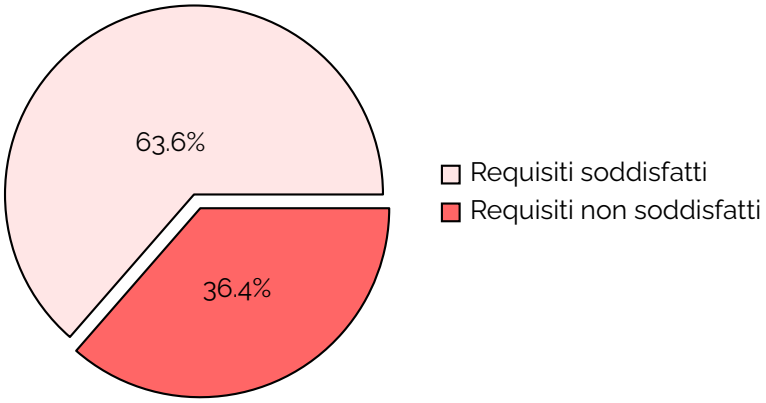

Figure 44: Percentuale di requisiti funzionali opzionali soddisfatti

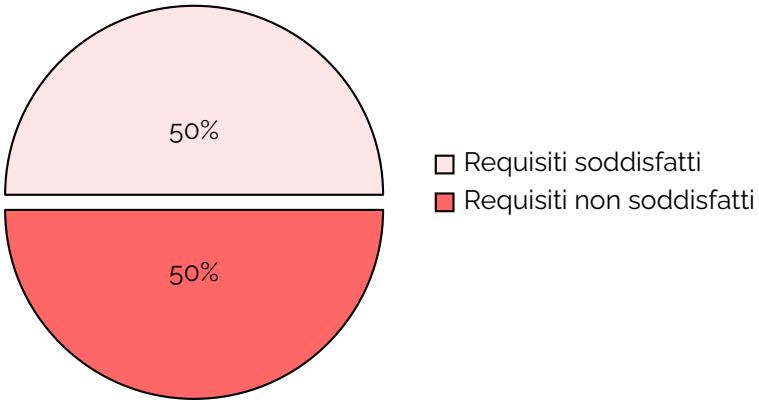

Figure 45: Percentuale di requisiti funzionali desiderabili soddisfatti